CODE AUDIT REPORT

GnB-SMH Source Code Audit

Ultra-deep security and consensus-consistency review, with a comparative analysis against Bitcoin Core

GnB (Gödel and Bell) post-quantum proof-of-work BlockDAG · code-only assessment · 14 October 2026

Item	Value
Subject	GnB-SMH (Gödel and Bell): post-quantum proof-of-work BlockDAG, repository `gnb-smh/gnb-smh`
Commit audited	`125e9e51d09347f474ddc17fc862d66de69498f6` (main, 2026-10-10)
Comparison baseline	Bitcoin Core master `f13ae519de94394314cd63a27924b0998ac65d45` (2026-10-14)
Report date	14 October 2026
Scope	All Rust crates (node, gnb-crypto, gnb-smh, coherence-core, gnb-wallet-wasm, vendored pqcrypto-internals and libp2p-yamux), the standalone `rust/` crate, Python references, web wallet, explorer, Worker, deployment scripts and CI
Method	Line-by-line manual review in seven areas; execution of every test suite; independent consistency harness against Bitcoin Core, BCHN and rust-bitcoin oracles; mutation testing; dependency audit
Explicitly excluded	Hashrate, adoption, market, decentralisation and any property that depends on network participation
Toolchain	rustc 1.94.1; cargo-audit 0.22.2 (RustSec DB 2026-10-14); cargo-deny 0.20.2; cargo-mutants 27.1.0; proptest 1.11; rust-bitcoin 0.32; Python 3.13
Prepared with	Claude Code (AI-assisted code audit). Findings are code observations with file:line evidence; they are not a guarantee of absence of other defects.

Contents

1. Executive summary
2. Scope, methodology and limitations
3. System overview and code inventory
4. Test execution and tooling results
5. Independent consistency harness
6. Findings
7. Component comparison with Bitcoin Core
8. Historical Bitcoin bug classes applied to GnB
9. Comparative security scorecard
10. Remediation roadmap
Appendix A. GnB test-module inventory
Appendix B. Bitcoin Core security and consistency test modules referenced
Appendix C. Reproduction commands

1. Executive summary

Verdict. At commit `125e9e5` GnB-SMH is not safe to secure value on a public network. The audit confirmed **5 Critical** and **19 High** findings, all five re-verified in the code by the lead auditor and each demonstrated by an executable proof-of-concept test. On a weighted, code-only scorecard GnB-SMH scores **3.1 / 10** against **9.1 / 10** for Bitcoin Core.

The cryptographic and arithmetic building blocks are good and, in places, exemplary; the failures are in how blocks and transactions flow through the node: what state a block is validated against, what a peer can make a node do before any proof of work is checked, and what the miner assembles.

3.1 / 10	9.1 / 10
GnB-SMH (commit 125e9e5)	Bitcoin Core (master f13ae51)

What is strong

- **Proof-of-work arithmetic is exact.** The compact-target codec accepts and decodes exactly what Bitcoin Core accepts across all 587,202,560 values with a decodable exponent; ASERT is bit-identical to the aserti3-2d fixed-point construction on more than 350,000 cases; GnB-SMH in Rust equals the normative Python definition on 103 inputs; timestamps are bounded so that timewarp is impossible (better than Bitcoin mainnet today).
- **Signatures are combined correctly.** The hybrid scheme requires both ML-DSA-65 and Falcon-1024, parses every component at exact length, uses the constant-time integer Falcon variant, fails closed on randomness errors, and binds the sighash to a domain and a chain id.
- **Arithmetic hygiene and memory safety.** Value sums use checked arithmetic throughout (CVE-2010-5139, CVE-2012-2459 and CVE-2018-17144 classes are covered); consensus crates forbid `unsafe`; production code has 3 `unsafe` sites, all FFI; release builds keep overflow checks.
- **The existing tests pass.** 1,040 test executions in 61 binaries, the standalone crate, the vendored guards (in isolation) and the Python references all pass. The ledger-v1 sparse Merkle tree prototype showed no soundness defect.

What breaks

- **Consensus state model (DAG-1, VAL-2, VAL-3, DAG-6).** Every block, including parallel DAG blocks, is validated against the node's current tip UTXO set, so two honest nodes that see two sibling blocks containing the same mempool transaction in opposite order reject each other's branch forever. Duplicate coinbases, same-block coinbase spends and a node-local reorg cap add miner-triggerable splits.
- **Unauthenticated network denial of service (POW-2, NET-2).** One gossip frame with `height = u64::MAX` panics the block processor on every node it reaches; junk blocks are relayed before validation and each costs every node a full 64 MiB proof-of-work evaluation (~0.8 s) against attacker-chosen bits, with no penalty.
- **Block production liveness (MEM-1, MEM-2).** A single gossiped dust transaction, or about 116 ordinary pending transactions, makes every template produced by the stock miner and `getblocktemplate` consensus-invalid.
- **Assurance.** CI never compiles or tests the node workspace, there is no fuzzing harness, one test file is compiled out, feature unification disables the real ASERT path in workspace tests, and the dependency policy fails its own checks.

Severity	Count	Meaning
Critical	5	Network-wide consensus failure or zero-cost network-wide halt reachable without special privileges
High	19	Miner- or peer-triggerable split, liveness failure, significant DoS, or realistic loss of funds
Medium	12	Exploitable under specific conditions or with bounded impact
Low	24	Defence in depth, latent defects, hygiene
Info	10	Observations, inactive components, documentation drift

2. Scope, methodology and limitations

The audit covers the complete repository at commit `125e9e5` and compares it with Bitcoin Core master at commit `f13ae51`. It is **code-only**: findings and scores are based on source code, tests and tool output. Nothing in this report depends on hashrate, miner or user adoption, market conditions, decentralisation, or other properties that only exist once a network is used; where an attack needs proof of work, the report states the work required by the code (for example, one evaluation at the pow limit) rather than its cost in hardware.

Method

- **Manual review.** Seven reviewers each read every line of one area: proof of work and difficulty; GhostDAG, chain selection and reorganisation; block and transaction validation, UTXO set and storage; cryptography and signatures; P2P, sync, RPC and explorer; wallets and web front end; mempool, shielded pool, attestation, ledger v1 and deployment. Each finding is traced to `file:line` and labelled *Confirmed by test*, *Confirmed by reading* or *Plausible*.
- **Verification.** Every Critical finding and the High findings that drive the verdict were re-read in the code by the lead auditor; duplicates reported by several reviewers were merged and are marked as independently reproduced. Severities were calibrated across areas.
- **Execution.** All of GnB's own suites were built and run in release mode, plus 73 proof-of-concept tests written during the audit (Appendix C explains how to run them; they live in `audit/poc` so that they do not alter the project's suites).
- **Independent consistency testing.** A separate crate, `audit/consistency`, applies Bitcoin Core's own vectors, line-for-line ports of Core and BCHN routines, and rust-bitcoin as oracles to GnB's consensus arithmetic (Section 5).
- **Mutation testing and dependency audit.** cargo-mutants on the PoW and difficulty crate; cargo-audit against the RustSec database of 14 October 2026; cargo-deny against the project's own policy.
- **Bitcoin Core comparison.** For each component the corresponding Core files, tests, fuzz targets and CI jobs at `f13ae51` were identified and counted directly from the source tree.

Severity scale

Severity	Definition used
Critical	Permanent divergence of honest nodes, or a network-wide halt or crash that any unauthenticated peer can trigger at negligible cost
High	A split or liveness failure that a miner or peer can trigger, a significant amplified DoS, a realistic path to loss of funds, or a missing assurance control that hides Critical defects
Medium	Exploitable under specific conditions, bounded impact, or a latent consensus risk
Low	Defence in depth, hygiene, latent defects in inactive code
Info	Observations without direct security impact

Scoring

The scorecard (Section 9) has nine categories weighted by their importance to the safety of a proof-of-work currency node. Each reviewer scored its component 0-10 for GnB and for the matching Bitcoin Core component; the lead auditor calibrated these against the findings and positive controls. Bitcoin Core was not re-audited: its scores reflect the code, tests and CI present at `f13ae51` and its known open issues (for example, the timewarp mitigation is not active on mainnet).

Limitations

- No live network was attacked. Network findings were demonstrated in-process or with local multi-node swarms.
- `accept_block` and the message processor live in the node binary (`main.rs`) and cannot be linked from tests. Two proof-of-concept files therefore contain a verbatim copy of that code, guarded by a test that checks the copy line by line against `main.rs`.
- No formal verification and no review of third-party crates beyond the advisory database and the FFI boundary of the post-quantum libraries.
- Timings were measured on a shared 4-core container and are indicative only.

3. System overview and code inventory

GnB is a hard fork of the Bloch BlockDAG node. It keeps Bloch's block format, GhostDAG consensus (k = 10), UTXO transaction model, hybrid ML-DSA-65 + Falcon-1024 signatures and libp2p network stack, replaces the proof of work with GnB-SMH (a sequential memory-hard SHAKE-256 function using 64 MiB per evaluation) and ASERT difficulty, and starts from a genesis that carries over the Bloch Genesis-3 ledger. A shielded pool, remote attestation and a stateless ledger (v1) are present but inactive in consensus.

Code base	Code lines	Comment lines	Files	Notes
GnB `node/src`	15,882	6,688	36	Full node, RPC, P2P, sync, storage; includes inline tests
GnB `crates/gnb-crypto/src`	9,170	2,410	19	Consensus types, signatures, wallet
GnB `crates/gnb-smh/src`	652	118	3	PoW and difficulty
GnB `crates/coherence-core/src`	549	202	1	Shielded-pool primitives (inactive)
GnB `crates/gnb-wallet-wasm/src`	188	27	1	Browser wallet core
GnB vendored Rust (`pqcrypto-internals`, `libp2p-yamux`)	623	331	2	Plus vendored PQClean C sources
GnB `rust/` standalone reference	1,270	126	3	PoW + ledger v1 prototype
GnB integration tests (`*/tests`)	8,769	2,552	56	Excluding audit proof-of-concept files
GnB Python reference	1,465	-	8	Normative PoW and ledger definitions
GnB production vs test lines (all Rust)	19,013 / 18,181	-	-	Non-blank, non-comment; inline `#[cfg(test)]` counted as test
Bitcoin Core `src` (excl. tests, bench, qt, vendored)	138,245	37,026	674	C++
Bitcoin Core `src/test` (unit, fuzz, util)	62,572	10,499	326	of which fuzz 20,399 lines in 152 files
Bitcoin Core `src/wallet/test`	5,004	821	31	
Bitcoin Core `test/functional`	73,085	-	331	Python, 285 test scripts
Bitcoin Core `src/secp256k1`	52,819	9,121	124	Vendored subtree with its own exhaustive tests

GnB is roughly a seventh of Bitcoin Core's size, but the BlockDAG adds machinery (coloring, reachability, mergesets, multi-parent templates) that a longest-chain node does not have. Bitcoin Core's test code exceeds its production code when functional tests are included; GnB's test lines are about equal to its production lines, but concentrated in unit tests.

Assurance assets

Assurance asset	GnB-SMH (125e9e5)	Bitcoin Core (f13ae51)
Unit and integration tests	101 modules, ~900 test functions (884 Rust, 18 Python incl. vectors); 1,040 executions in the workspace run	900 Boost test cases plus 113 secp256k1 test groups
Property-based tests	5 GhostDAG proptests; seeded random tests (mempool index, SMT, wire round trips)	Pervasive in fuzz targets; randomized unit tests
Coverage-guided fuzzing	None (5 seed corpora replayed by a stable-Rust test; referenced cargo-fuzz targets do not exist)	206 fuzz targets (libFuzzer/AFL++/honggfuzz) with external corpora
Functional / multi-node tests	A few live two-swarm tests; no framework	285 functional test scripts (63 p2p, 20 mempool, 68 wallet)
Data-driven vectors	7 PoW vectors, ledger vectors (28 blocks), 15 ACVP ML-DSA, 3 Falcon round-3, address vectors	tx_valid/tx_invalid, sighash.json (500), script_tests.json (1,258), key_io, base58, bip341 vectors
Sanitizers / dynamic analysis	None in CI	ASan+UBSan+integer, MSan, TSan, valgrind, fuzz with sanitizers

Assurance asset	GnB-SMH (125e9e5)	Bitcoin Core (f13ae51)
Static analysis in CI	None (no clippy, no cargo-deny, no cargo-audit job)	clang-tidy, IWYU, 15 lint scripts, thread-safety analysis
Cross-implementation reference	Python normative PoW and ledger; no reference for difficulty, GhostDAG or validation	Consensus is the implementation; extensive vectors
Invariant checkers	Reachability self-check at boot; integrity hash chain; mempool index check	`CheckBlockIndex`, `CTxMemPool::check`, `-checkblocks`

Panic and unsafe surface (production code)

Indicator (production code, excluding tests)	GnB-SMH	Comment
`unsafe` occurrences	3	2 FFI exports in the WASM wallet, 1 FFI randomness override; consensus crates use `#![forbid(unsafe_code)]`
`.unwrap()` / `.expect(`	48 / 54	Mostly metrics registration, start-up and tests of invariants; no attacker-reachable unwrap found by reviewers
`panic!` / `unreachable!` / `todo!`	3 / 0 / 0	In the shielded verifier stub
Integer `as` casts	263	Reviewed on consensus paths; one 32-bit overflow note (CRY-8)
`checked_*` / `saturating_*` calls	120	Consistent checked arithmetic on value sums
Release `overflow-checks`	on	Fail-closed on overflow, but turns unchecked arithmetic on peer data into panics (POW-2)
Bitcoin Core comparators	-	718 `Assert`, 571 `Assume`, 208 `CHECK_NONFATAL`, 2,620 `assert`, 1,696 thread-safety annotations

4. Test execution and tooling results

Every suite in the repository was executed. All pass; the defects in Section 6 exist because the suites do not exercise the relevant paths, not because tests fail.

Tool	Result
cargo test (workspace, release, locked)	61 binaries, 1,040 executions passed, 0 failed, 10 ignored (36 min; node module tests run under both the library and binary targets)
cargo test (`rust/` standalone)	18 passed, 0 failed
Vendored crates (isolated copies)	pqcrypto-internals 17 + 2 passed (1 ignored); libp2p-yamux 2 + 2 passed; cannot run in place (AUD-2)
Python reference	check_vectors.py 7/7; test_ledger.py 11/11; ledger vectors regenerate byte-identically
Audit consistency harness	35 tests in 7 modules, all passing (Section 5)
Reviewer proof-of-concept tests	73 tests in 9 files, all passing; each passing test demonstrates its finding
cargo-mutants 27.1.0 (gnb-smh)	222 mutants: 176 caught, 37 missed, 9 unviable (82.6%); the audit harness kills 15 of the 37; the other 22 are equivalent (12) or in non-consensus accessors and Display impls (10)
cargo-audit 0.22.2	11 vulnerabilities: quinn-proto x5 (lockfile only, not compiled), rustls 0.23.45 x4 (reachable), hickory-proto 0.25.2 x2 (reachable, ignored in deny.toml); 6 unmaintained warnings (bincode, paste, pqcrypto-falcon, pqcrypto-mldsa, pqcrypto-kyber, ...). The standalone `rust/` crate: none
cargo-deny 0.20.2	bans ok, sources ok, licenses FAILED (first-party AGPL crates not allowed); 5 unmatched and 2 unnecessary skips; 15 license entries never encountered
cargo clippy (workspace, all targets)	Fails: 1 deny-by-default lint (`absurd_extreme_comparisons` at node/src/consensus/mod.rs:687; the activation constant is 0, so the gate documented as `u64::MAX` is always on) plus 130 warnings before the build stopped (node 91, gnb-crypto 18 in library targets; mostly needless borrows, complex types, redundant closures). Clippy is not run in CI

Mutation testing of the PoW and difficulty crate

cargo-mutants generated 222 mutants of `crates/gnb-smh/src/{lib,difficulty}.rs`. The crate's own tests killed 176 of 213 viable mutants (82.6%). Among the 37 survivors were consensus-relevant changes: the strict PoW comparison (`<` to `<=`), `verify` and `hash` returning constants, `work` and a schedule off-by-one. Re-running the survivors against the audit harness killed 15 more, including all of those; the remaining 22 are equivalent mutants (bit-disjoint `|` vs `^`, zero-iteration loop bounds, clamp-on-equality) or mutate non-consensus accessors and `Display` implementations. With the harness, every non-equivalent consensus mutant in the crate is detected.

5. Independent consistency harness

`audit/consistency` is a stand-alone crate (its own workspace, so the project's manifests and lockfile are untouched) that treats Bitcoin Core, BCHN and rust-bitcoin as reference implementations of the arithmetic GnB inherits from them. It uses `proptest` for property-based cases, `bitcoin` 0.32 (rust-bitcoin) as a crate oracle, `primitive-types` for 256/512-bit oracles, and the normative Python definition of GnB-SMH. All 35 tests pass.

Module (tests)	Oracle	Result
compact_core_vectors.rs (3)	Bitcoin Core `bignum_SetCompact` (20 vectors) and `CheckProofOfWork_test_*` rejections	GnB accepts exactly the values Core's `DeriveTarget` accepts, decodes them to the same integers and re-encodes like `GetCompact`
compact_differential.rs (5)	Exhaustive sweep of every compact value with exponent 0..=34 (587,202,560 values) plus a sample of every larger exponent; 200,000-case proptests against Core `GetCompact` and rust-bitcoin `Target::to_compact_lossy` / `from_compact`	Zero differences. 268,435,424 encodings are accepted by both; 17,694,689 of them are non-canonical, which is safe only because consensus compares `bits` to `expected_bits` as a u32
asert_differential.rs (8)	Port of the BCHN `CalculateASERT` fixed-point step (512-bit, no truncation) with GnB's anchor convention; 100,000-case proptests on real and random anchors; 152,358-point deterministic sweep	Compact bits identical, full 256-bit targets identical; monotone in time; bounded to [1, pow_limit]; exact doubling per half-life; no panic on any (height, timestamp). Found AUD-1 (saturation for anchors near 2^256)
merkle_consistency.rs (4)	rust-bitcoin `merkle_tree::calculate_root` and a port of Core `ComputeMerkleRoot`	Roots identical; CVE-2012-2459 collision reproduced (GnB relies on its duplicate-transaction rule); block id unchanged when only signature bytes change
amount_consistency.rs (5)	CVE-2010-5139 construction; closed-form schedule	Overflow detected; total mining issuance 899,999,851,392,253 sat of 900,000,000,000,000 allocated (1.486 GNB never issued by rounding); fee function total over usize
pow_reference_differential.rs (3)	Normative `reference/gnb_smh.py`	96 random cases (header lengths 0-399, any u64 nonce, log2_n 1-12) and one canonical (2^19) case identical; reused and per-thread scratchpads leak no state
pow_check_semantics.rs (7)	Core `CheckProofOfWorkImpl` and `GetBitsProof`	GnB's strict `out < target` pinned (Core accepts `hash == target`); `work()` never overstates and is within 2^-64 of Core for every reachable target, but differs from the Python specification (POW-6)

The harness is also the reason two findings exist: it measured the `work()` deviation from the specification (POW-6) and found the ASERT saturation for anchors near the pow limit (AUD-1). It should be adopted in CI; it runs in about 30 seconds in release mode.

6. Findings

Findings are ordered by severity. Each lists its location, the evidence status, a concrete failure scenario where applicable, the corresponding Bitcoin Core control, and a recommendation. Identifiers name the review area: POW (proof of work), DAG (GhostDAG and reorg), VAL (validation and storage), CRY (cryptography), NET (P2P, sync, RPC), WAL (wallet and web), MEM (mempool), COH (shielded pool), LED (ledger v1), ATT (attestation), OPS (CI and operations), AUD (found by the audit harness).

Severity	Count
Critical	5
High	19
Medium	12
Low	24
Info	10
Total	70

Sev.	ID	Title	Area
Critical	DAG-1	Fork and parallel blocks are validated against the selected-chain UTXO set: honest nodes split permanently in ordinary operation	Consensus / GhostDAG
Critical	MEM-1	A single gossiped dust transaction halts block production on every node that mines from its mempool	Mempool / mining
Critical	MEM-2	Block templates have no byte budget: ~116 ordinary pending transactions make every template exceed MAX_BLOCK_SIZE	Mempool / mining
Critical	NET-2	Zero-cost network-wide CPU exhaustion: blocks are relayed before validation, the 64 MiB PoW runs on self-declared bits before cheap checks, and invalid senders are never penalised	P2P / block ingress
Critical	POW-2	One unauthenticated gossip frame panics the block-processing task (integer overflow on a wire-supplied height), repeatable network-wide	P2P / block ingress
High	DAG-11	The extension commit path is not atomic and write errors are ignored	Storage / crash consistency
High	DAG-2	More than 256 tips halts all mining that uses the stock template code	Mining / GhostDAG
High	DAG-3	Fork choice ignores merged blue work, and merged blocks never reach the ledger	Consensus / GhostDAG
High	DAG-6	Reorg depth cap and finality are node-local and history-dependent: nodes can diverge permanently	Consensus / reorg
High	DAG-7	Unbounded per-block cost: O(past) mergeset computation, full-subtree reachability reindex, no mergeset or merge-depth limit	Consensus / DoS
High	MEM-3	Mempool is never conflict-swept, revalidated after reorgs, or expired; one double-spend wedges the local miner	Mempool
High	MEM-4	Gossip admission checks coinbase maturity against the DAG block count instead of the chain height	Mempool
High	MEM-5	Transaction admission performs O(inputs x size) signature work before cheap size, duplicate and standardness checks, with no peer penalty	Mempool / P2P
High	NET-4	Honest relays are penalised for frames they merely forwarded, cutting honest paths	P2P
High	NET-5	A requester penalises an honest server for rate-limiting it	P2P / sync
High	NET-6	One Version frame latches IBD mode network-wide and pauses mining	P2P / sync
High	NET-7	Gossip requests produce mesh-wide broadcast replies (O(N^2) amplification)	P2P
High	OPS-1	CI never builds or tests the consensus workspace; documented guards and fuzz targets do not exist	CI / release engineering
High	POW-3	Near-free blocks on deep forks: absolute-schedule ASERT, a pow limit of ~2^256 and no anti-DoS work rule	Difficulty / DAG
High	POW-4	Orphan pool admission is gated only by self-declared bits: memory exhaustion and eviction of honest orphans	P2P / orphan pool

Sev.	ID	Title	Area
High	VAL-2	No BIP30/BIP34 protection: duplicate coinbase txids overwrite UTXOs and rollback deletes the earlier block's coin	Validation / UTXO
High	VAL-3	A block may spend its own coinbase (maturity bypass), while the reorg path forbids it: miner-triggerable split	Validation
High	WAL-1	Wallets sign with UTXO values supplied by the node/API and have no maximum-fee guard	Wallet
High	WAL-2	The Cloudflare Worker reaches the explorer origin over plain HTTP with the shared secret, and caches per origin headers	Web infrastructure
Medium	CRY-1	Consensus accepts third-party witness malleation and nothing commits to script_sig bytes	Signatures / block format
Medium	DAG-10	accept_block is not serialized across ingress paths	Consensus / concurrency
Medium	DAG-4	The live (fast) GHOSTDAG coloring violates the k-cluster rule	Consensus / GhostDAG
Medium	NET-10	Unbounded frontier state and fan-out from unsolicited frames	P2P / sync
Medium	NET-11	Weak peer exchange and eclipse resistance	P2P / peer management
Medium	NET-12	The public explorer chain allows unbounded full-chain scans	RPC / explorer
Medium	NET-8	Rate-limit and penalty maps never evict	P2P
Medium	NET-9	A single attacker can capture and stall all IBD pulls	P2P / sync
Medium	OPS-2	Dependency policy is stale and not enforced; reachable advisories are ignored	Supply chain
Medium	POW-5	Consensus difficulty code is weakly tested: feature unification disables the real ASERT path in workspace tests, 37 mutants survive, no external oracle	Difficulty / tests
Medium	WAL-3	Explorer and landing pages have no CSP and render API data through innerHTML without escaping	Web wallet / explorer
Medium	WAL-4	Explorer API is exhaustible by idle connections and is not firewalled to the proxy	Explorer API
Low	AUD-1	ASERT saturates its intermediate product before a negative shift: up to ~2x too hard for anchors near the pow limit	Difficulty (latent)
Low	AUD-2	The vendored crates' own guard tests cannot be run in place	Supply chain / tests
Low	COH-2	Shielded state is mutated before the block's disposition is known and never rolled back	Shielded pool (inactive)
Low	CRY-2	ML-DSA-only suite 0x0002 is live without activation and hybrid signatures are separable	Signatures
Low	CRY-3	Sighash preimage is defined by a serialization library and does not commit to spent amounts	Signatures
Low	CRY-4	Secret keys, mnemonics and passwords are not reliably zeroized	Keys / memory hygiene
Low	CRY-5	Falcon signer return code is discarded and signatures are not verified after signing	Signatures
Low	CRY-7	Address checksum and the default keystore format do not bind the network	Addresses / keystores
Low	DAG-5	Restart re-marks interior blocks as tips	Consensus / persistence
Low	DAG-8	DAG height and header height disagree	Consensus
Low	MEM-6	Policy gaps: no rolling minimum fee, no replacement policy, no package/orphan handling; fee estimate defaults below the consensus minimum	Mempool
Low	MEM-7	Reorg re-injection skips maturity and minimum-fee re-checks	Mempool / reorg
Low	NET-13	RPC method strings become unbounded Prometheus labels and log lines	RPC / metrics
Low	NET-14	The node exits if the metrics or RPC server fails to bind	Node lifecycle
Low	NET-15	Explorer calls hold the DAG read lock across storage reads	RPC / explorer
Low	NET-16	Secret file and authentication hygiene	Secrets / RPC auth
Low	OPS-3	Systemd units lack most sandboxing directives; installer quoting issues	Deployment
Low	POW-6	work() deviates from the normative definition floor(2^256 / target)	Difficulty
Low	POW-7	The difficulty input is documented as the selected parent but is the latest timestamp over all parents; two notions of height coexist	Difficulty / documentation
Low	POW-8	Wire metadata (height, blue_score) is not committed by the block id; blue_score is never validated	Block format
Low	VAL-10	Merkle tree has no leaf/inner-node domain separation and no 64-byte transaction rule	Block format
Low	VAL-12	Script size limits live only in the wire parser; block size uses a different encoding than fees	Validation

Sev.	ID	Title	Area
Low	WAL-5	WASM wallet core is loaded without an integrity check and the committed binary is not reproducibly tied to source	Web wallet supply chain
Low	WAL-6	Coin selection and fee handling: smallest-first greedy selection, size-blind fees, silent dust-to-fee	Wallet
Info	ATT-1	Remote attestation is not used for consensus or peer trust and fails closed	Attestation (inactive)
Info	COH-1	Shielded spend statement does not bind spending authority (known CR-01); unreachable today	Shielded pool (inactive)
Info	CRY-6	Unused cryptographic dependencies are compiled (blst, secp256k1)	Dependencies
Info	CRY-8	Minor: 32-bit overflow in script parsing, message-signing domain without chain id, stale comments	Signatures
Info	DAG-12	Minor GhostDAG observations	Consensus
Info	LED-1	Ledger v1 sparse Merkle tree: no soundness defect found; witnesses uncommitted and coinbase uniqueness not required	Ledger v1 prototype
Info	NET-17	Dead or diverging networking code	P2P
Info	POW-9	Minor PoW observations	PoW
Info	VAL-11	No MoneyRange check and no supply-sum audit; the 10 M GNB cap is implied, not enforced	Monetary policy
Info	WAL-7	Wallet design notes: no mnemonic backup for default wallets, network chosen by file name, HTTP-only RPC client	Wallet UX / transport

Detailed findings

Critical DAG-1 — Fork and parallel blocks are validated against the selected-chain UTXO set: honest nodes split permanently in ordinary operation

Location: node/src/main.rs:2054-2066,2096; node/src/main.rs:2706-2721,2824-2884; node/src/main.rs:2203-2211; docs/LEDGER.md (section 7) · Area: Consensus / GhostDAG · Evidence: Confirmed by test (audit_poc_dag.rs, mirrored accept_block) and by reading (lead auditor re-verified); independently reproduced by the validation reviewer

`accept_block` validates every transaction of every incoming block, including blocks that will only be fork-losers or merged siblings, by resolving inputs through `store.get_utxo`, i.e. the node's single UTXO set at its current selected tip. Any failure rejects the whole block before it enters the DAG, and nothing remembers it, so it is re-rejected each time it is fetched. Validity therefore depends on what the node already applied, i.e. on arrival order. This contradicts docs/LEDGER.md section 7, which says a conflicting transaction in a parallel block becomes a rejected transaction, not an invalid block.

Failure scenario. Two miners on the same tip both include mempool transaction T (siblings a1 and b1). Node X sees a1 first, spends T's input, and rejects b1; node Y does the opposite. X's descendants of a1 become orphans at Y, Y fetches a1 and rejects it again, forever, no matter how much work accumulates on X's side. The same happens to any fork block that spends an output created earlier on its own fork.

Bitcoin Core. `AcceptBlock` stores side-branch blocks after context-free and contextual header/block checks only; UTXO validation happens in `ConnectBlock` when `ActivateBestChainStep` connects the branch after `DisconnectTip` unwinds to the fork point (src/validation.cpp).

Recommendation. Separate acceptance from connection: admit DAG blocks after stateless and contextual checks, validate transactions only when a block is connected to the selected chain or merged (with the section 7 mergeset rules), and cache invalidity only for permanent failures.

Critical MEM-1 — A single gossiped dust transaction halts block production on every node that mines from its mempool

Location: node/src/main.rs:1234-1256; node/src/main.rs:2689-2762; node/src/mempool/mod.rs:148-273; node/src/rpc/mod.rs:1717-1732; crates/gnb-crypto/src/core/mod.rs:982-991 · Area: Mempool / mining · Evidence: Confirmed by test (template level) and by reading (gossip path); independently reproduced by the validation reviewer

Consensus rejects any block containing a non-coinbase output with `0 < value < 546` (`Block::validate_dust`, called from `validate_structure`). The RPC submission path applies the same dust rule at admission, but the gossip path (`NetworkMessage::NewTransaction` -> `validate_tx_standalone` -> `Mempool::add`) does not. A dust transaction is therefore admitted from any peer, never expires, is re-broadcast by the node itself every ~30 s, and is selected by fee rate into every template (`get_for_block(2000)`). Every self-mined or `getblocktemplate` block then fails `validate_structure`; the miner only logs `miner bug` and retries with the same template.

Failure scenario. An unauthenticated peer gossips one transaction with a 1-sat output and a high fee rate. All honest nodes admit it and every template becomes consensus-invalid. The attacker pays nothing because the transaction can never be mined.

Bitcoin Core. `MemPoolAccept::PreChecks` (src/validation.cpp) runs `IsStandardTx` (dust, weight) before any script work on every ingress path, and the miner validates its own template with `TestBlockValidity` (src/node/miner.cpp).

Recommendation. Use one standardness function for every ingress path (gossip, RPC, reorg re-injection) that is a superset of the block rules; validate templates with the block rules before hashing and evict offending transactions automatically.

Critical MEM-2 — Block templates have no byte budget: ~116 ordinary pending transactions make every template exceed MAX_BLOCK_SIZE

Location: node/src/mempool/mod.rs:310-325; node/src/main.rs:1618; node/src/rpc/mod.rs:1141; crates/gnb-crypto/src/core/mod.rs:82,1012 · Area: Mempool / mining · Evidence: Confirmed by test (audit_poc_mempool.rs); independently reproduced by the validation reviewer

`Mempool::get_for_block(limit)` caps templates by transaction count (2,000) only. A one-input hybrid ML-DSA-65 + Falcon-1024 transaction is about 8.6 kB, so 120 ordinary transactions produce a 1,035,636-byte template that `validate_structure` rejects as `block too large`. Three 390 kB transactions do the same.

Failure scenario. Normal load, or an attacker paying ~120 minimum fees, stops the internal miner and every `getblocktemplate` consumer from producing valid blocks until the mempool drains, which it cannot do because nothing is mined.

Bitcoin Core. `BlockAssembler` (src/node/miner.cpp) packages by fee rate under a weight budget (`nBlockMaxWeight`) and calls `TestBlockValidity` before returning the template.

Recommendation. Select by fee rate under a byte (or weight) budget that reserves space for the header and coinbase; add a template self-check.

Critical NET-2 — Zero-cost network-wide CPU exhaustion: blocks are relayed before validation, the 64 MiB PoW runs on self-declared bits before cheap checks, and invalid senders are never penalised

Location: node/src/network/mod.rs:670-690,1050-1064; node/src/main.rs:1133,1852,1890,1985-2052; crates/gnb-crypto/src/core/mod.rs:907-932,1010-1019; docs/CONSENSUS.md:33-54 · Area: P2P / block ingress · Evidence: Confirmed by test (785-826 ms per invalid-PoW block in release, no penalty; three reviewers independently: POW-1, VAL-7, NET-2; lead auditor confirmed no `validate_messages` / `report_message_validation_result` call exists)

Gossipsub is built without `validate_messages()`, so libp2p forwards every frame to the mesh before the application sees it, and `report_message_validation_result` is never called, so the configured invalid-delivery penalties never fire. Each node then runs `validate_structure()`, which ends with the full GnB-SMH evaluation against the header's own `bits`, inline on the single message-processing task and before the cheap contextual checks (`expected_bits`, known parents, height) in `accept_block`. Failures are not cached. `docs/CONSENSUS.md` states the opposite ordering.

Failure scenario. An unauthenticated peer publishes junk `NewBlock` frames (random parents, valid Merkle root, any nonce). About 1.3 frames per second saturate the processor of every node in the mesh, and the bounded ingest queue then drops honest blocks. With `bits = 0x2100ffff` the junk even passes and is inserted into the PoW cache.

Bitcoin Core. Blocks and transactions are relayed only after validation; the PoW check is a cheap SHA-256d done first; invalid data leads to `Misbehaving`/discouragement (src/net_processing.cpp); headers presync and `MinimumChainWork` gate any expensive work.

Recommendation. Enable `validate_messages()` and report Accept/Reject; check expected bits, height and parent presence before PoW; remove PoW from `validate_structure`; apply per-peer PoW budgets and disconnects; run evaluations on a bounded worker pool.

Critical POW-2 — One unauthenticated gossip frame panics the block-processing task (integer overflow on a wire-supplied height), repeatable network-wide

Location: node/src/main.rs:1074-1078,1103-1130; node/src/network/mod.rs:83-88; node/src/main.rs:766-767,1296-1316; Cargo.toml:38 · Area: P2P / block ingress · Evidence: Confirmed by test (overflow on the exact expression) and by reading (reachability; lead auditor re-verified); independently reproduced by the networking reviewer

`NetworkMessage::NewBlock.height` is only required to equal `block.height`, and neither is committed to the block id or the PoW preimage. The backfill-flood guard then evaluates `height + core::CHECKPOINT_DEPTH < tip_h` before any proof-of-work check. Release builds set `overflow-checks = true`, so `height = u64::MAX` panics as soon as the tip is above 1,000 blocks (already true on mainnet). The supervisor catches the panic, wipes the orphan pool and restarts the loop after an exponential backoff that is never reset (cap 30 s); frames arriving meanwhile are dropped from the bounded queue.

Failure scenario. An attacker builds any block (no valid PoW needed), sets both heights to `u64::MAX`, and publishes it. Gossipsub forwards it to the whole mesh before application validation (MEM-5), so every node's processor panics. Re-sending a byte-mutated copy every ~30 s keeps block processing stopped on all nodes.

Bitcoin Core. Core derives height from the block index rather than trusting peer metadata, does arithmetic on validated values, and punishes malformed messages (`Misbehaving`, src/net_processing.cpp); its fuzz targets (`process_message`, `process_messages`) exercise this class of input continuously.

Recommendation. Drop the wire height (derive it from parents) or reject heights above `tip + slack` before any arithmetic; use checked/saturating arithmetic on every peer-supplied integer; reset the backoff after a healthy period; add a regression test and a fuzz target for `NewBlock` handling.

High DAG-11 — The extension commit path is not atomic and write errors are ignored

Location: node/src/main.rs:2230-2238,2353,2355; node/src/storage/mod.rs:645-647; node/src/reorg.rs:813-892 · Area: Storage / crash consistency · Evidence: Confirmed by test (simulated crash and restart) and by reading

An apply error is logged and processing continues; the DAG, integrity and reachability persists and `tip_hash` are `.ok()`-ignored; UTXO effects land before CF_DAG. A crash in between leaves applied effects for a block not in the DAG, which is then rejected on re-receipt (inputs already spent). A missing parent integrity record is zero-filled, leading to a fatal boot error, and there is no start-up reconciliation. The reorg path, by contrast, uses one atomic WriteBatch.

Bitcoin Core. `FlushStateToDisk` with head-block markers and `ReplayBlocks` recovery at start-up (src/validation.cpp).

Recommendation. Commit UTXO effects, undo data, DAG data and tip in one WriteBatch (as the reorg path does), propagate errors, and reconcile at start-up.

High DAG-2 — More than 256 tips halts all mining that uses the stock template code

Location: node/src/main.rs:1560,1654,1715-1732; node/src/rpc/mod.rs:1108; crates/gnb-crypto/src/core/mod.rs:366,1011 · Area: Mining / GhostDAG · Evidence: Confirmed by test (301 tips: template rejected with `too many parents`; wire decode fails)

The internal miner and `getblocktemplate` use every bodied tip as a parent, but `validate_structure` and the wire decoder reject more than `MAX_PARENTS = 256`. Nobody using the stock code can merge the tips, so the condition persists. Leaf blocks are cheap to create on deep forks (POW-3).

Bitcoin Core. Not applicable (single parent); Kaspa bounds parents (10) and mergeset size.

Recommendation. Select at most `MAX_PARENTS` parents by blue work, and add mergeset-size and merge-depth limits.

High DAG-3 — Fork choice ignores merged blue work, and merged blocks never reach the ledger

Location: node/src/consensus/mod.rs:1062-1066,1153-1167,1426-1440; node/src/reorg.rs:161-224; node/src/main.rs:2203-2243 · Area: Consensus / GhostDAG · Evidence: Confirmed by test (a tip with 15 blue blocks in its past loses to a bare 11-block chain)

`blue_work = selected_parent.blue_work + own work`; the comment calls this Kaspa-aligned, but GHOSTDAG/Kaspa add the work of every blue block in the mergeset. UTXO state is applied only along the selected-parent chain, so transactions and coinbases of merged blue or red blocks are never applied. The DAG coloring therefore affects neither fork-choice weight nor the ledger: honest parallel blocks add no weight against an attacker chain, and their miners are not paid.

Bitcoin Core. Bitcoin's most-work rule counts every block on the chain; there is no merged-but-ignored work.

Recommendation. Compute blue work over all mergeset blues and implement the mergeset transaction semantics of docs/LEDGER.md section 7 (or document the chain as selected-chain-only and stop advertising GHOSTDAG security).

High DAG-6 — Reorg depth cap and finality are node-local and history-dependent: nodes can diverge permanently

Location: node/src/reorg.rs:42,808-814; node/src/main.rs:1525-1532,1924-1929,2192-2200,2384-2394 · Area: Consensus / reorg · Evidence: Confirmed by test (node X refuses a 1,101-block reorg; node Z, fed the same blocks interleaved, follows the heavier fork)

`MAX_REORG_DEPTH = 1000` and the finality gate use each node's own history. A node that saw the honest chain first refuses a heavier fork deeper than the cap, while a node syncing fresh, or on the other side of a partition, follows it. There is no invalidate/reconsider tool, and `FINALITY_FROZEN`, documented to allow a heavier chain to deep-reorg the node back, is defeated by the independent cap.

Bitcoin Core. No reorg-depth limit and no checkpoints on master; operators have `invalidateblock`/`reconsiderblock` (src/validation.cpp `InvalidateBlock`, `PreciousBlock`).

Recommendation. Replace the local cap by an objective DAG rule (finality point / bounded merge depth evaluated from the DAG itself) and add operator recovery commands.

High DAG-7 — Unbounded per-block cost: O(past) mergeset computation, full-subtree reachability reindex, no mergeset or merge-depth limit

Location: node/src/consensus/mod.rs:417,1326-1364; node/src/consensus/reachability.rs:233-297; node/src/main.rs:1103-1130 · Area: Consensus / DoS · Evidence: Confirmed by test (timings) and by reading (guard bypass)

`compute_mergeset` always walks `compute_past(selected_parent)`, the reachability reindex recomputes subtree sizes per child, `candidate_anticone_blues` walks the selected chain with no k+1 early exit, and everything runs under the global DAG write lock. Measured at 12,000 blocks (release, loaded host): tip extension 5.8-6.2 ms; deep side-block insertion 1.1-4.4 s; one genesis sibling 6.2-10.2 s; one merge block with a 1,000-block mergeset 7.2-10.2 s. The backfill guard can be bypassed by making the deep block `awaited` through a crafted orphan.

Bitcoin Core. Header-first sync with anti-DoS work thresholds; no per-block cost proportional to history.

Recommendation. Use the reachability index for incremental past/mergeset computation, add a mergeset-size limit, a merge-depth bound, k+1 early exit, stored subtree sizes and a reindex window.

High MEM-3 — Mempool is never conflict-swept, revalidated after reorgs, or expired; one double-spend wedges the local miner

Location: node/src/mempool/mod.rs:347-366; node/src/main.rs:2241,2311; node/src/reorg.rs:894-905 · Area: Mempool · Evidence: Confirmed by test (mempool) and by reading; independently reproduced by the validation reviewer

`remove_confirmed` removes only transactions whose txids appear in a new block. A pooled transaction whose input was spent by a different confirmed transaction, or whose input disappears in a reorg, stays in the pool indefinitely and keeps being selected into templates, which `accept_block` then rejects with `UTXO not found`.

Failure scenario. Any ordinary double-spend or fee bump in which the competing transaction confirms first wedges the miner of every node that saw the losing version, until restart.

Bitcoin Core. `CTxMemPool::removeForBlock` -> `removeConflicts`, `MaybeUpdateMempoolForReorg`/`removeForReorg`, `Expire` (336 h default), `TrimToSize` and the `CTxMemPool::check` consistency audit (src/txmempool.cpp).

Recommendation. On every connected block remove conflicting transactions; on reorg revalidate the pool against the new tip; add expiry and a periodic consistency check.

High MEM-4 — Gossip admission checks coinbase maturity against the DAG block count instead of the chain height

Location: node/src/main.rs:1239; node/src/main.rs:2891-2908; node/src/consensus/mod.rs:1523-1525; node/src/rpc/mod.rs:955-963 · Area: Mempool · Evidence: Confirmed by reading; also reported by the validation reviewer

The gossip path passes `dag.block_count()` (every DAG block, including merged/red blocks) as `current_height` to `validate_tx_standalone`. In a BlockDAG the block count exceeds the selected-chain height (the RPC path's own comment records a 10,769-block gap), so immature coinbase spends pass the gossip check while `accept_block` correctly rejects them using `block.height`.

Failure scenario. Any coinbase owner gossips a spend of an immature coinbase; it enters every pool and poisons templates for up to 100 blocks (same wedge as MEM-1/MEM-3).

Bitcoin Core. Core checks maturity with `CheckTxInputs` against the active chain height on every path (src/consensus/tx_verify.cpp).

Recommendation. Use the selected-tip height and the shared `core::check_coinbase_maturity` on all ingress paths; add a regression test at the gossip boundary (today `validate_tx_standalone` lives in the binary and is untested).

High MEM-5 — Transaction admission performs O(inputs x size) signature work before cheap size, duplicate and standardness checks, with no peer penalty

Location: node/src/main.rs:1234-1256; crates/gnb-crypto/src/core/mod.rs:572-612,645-663; node/src/mempool/mod.rs:164,195; node/src/network/mod.rs:665-690 · Area: Mempool / P2P · Evidence: Cost confirmed by test (57 ms -> 2,940 ms for 30 -> 235 inputs); end-to-end plausible

Signature verification runs before the 400 kB `MAX_TX_SIZE` and duplicate checks. `Transaction::sighash` deep-clones and re-encodes the whole transaction per input, so cost grows quadratically. Gossipsub is configured without application-level validation (`validate_messages()`), so frames are forwarded before checking; `from_wire_bytes` accepts trailing bytes, which defeats message-id de-duplication; invalid transactions only produce a debug log and there is no recent-rejects filter.

Failure scenario. An attacker holding ~235 UTXOs signs one 4 MB transaction that is never admitted (oversize), so the inputs are never spent, and replays byte-mutated copies; each copy costs every node about 3 s on the single message-processing task, delaying or dropping block frames from the shared 1,000-slot queue.

Bitcoin Core. `PreChecks` enforces size and standardness and rejects `txn-already-in-mempool` before `PolicyScriptChecks`; BIP143/BIP341 sighash precomputation makes hashing linear; misbehaving peers are scored and disconnected; `m_lazy_recent_rejects` drops repeats (src/node/txdownloadman_impl.cpp).

Recommendation. Order checks cheap-to-expensive (size, standardness, duplicates, UTXO presence, fee, then signatures); cache sighash midstates; reject trailing bytes; validate gossip before forwarding and penalise peers; add a rolling reject filter.

High NET-4 — Honest relays are penalised for frames they merely forwarded, cutting honest paths

Location: node/src/network/mod.rs:394-452,1050-1064 · Area: P2P · Evidence: Confirmed by test (live 4-node topology)

Wire-bound penalties are charged to `propagation_source`, i.e. the neighbour that forwarded the frame, and the tracker never decays or evicts. Because gossip forwards before validation (NET-2), an attacker's malformed frames make honest nodes score their honest neighbours below the gossip threshold.

Failure scenario. Attacker A, honest C and D each connected only to hub B: five malformed frames from A make C and D score B at -200, prune it from the mesh and ignore its IHAVE; D's next block reaches B but never C, while TCP stays up.

Bitcoin Core. Misbehaviour is attributed to the peer that sent invalid data after validation, and discouragement is per peer.

Recommendation. Validate before forwarding and use gossipsub Reject so penalties follow the author; decay application scores.

High NET-5 — A requester penalises an honest server for rate-limiting it

Location: node/src/network/mod.rs:1544,1588-1594; node/src/network/sync_rr.rs:222-229 · Area: P2P / sync · Evidence: Confirmed by test (unit and live)

When `SyncRequestLimiter` refuses a request it drops the response channel; libp2p closes the stream cleanly, the requester reads an empty frame, decodes it as invalid data and applies a -100 protocol-violation penalty to the honest server.

Failure scenario. An ordinary IBD burst of 200 pulls makes the syncing node stop receiving the honest server's blocks.

Bitcoin Core. Explicit `notfound` responses and per-peer in-flight limits with stall timeouts (`MAX_BLOCKS_IN_TRANSIT_PER_PEER`, `BLOCK_STALLING_TIMEOUT`).

Recommendation. Send an explicit Busy response, treat EOF as a transport failure, and cap in-flight requests on the client side.

High NET-6 — One Version frame latches IBD mode network-wide and pauses mining

Location: node/src/network/mod.rs:1996-2007; node/src/sync/peer_state.rs:117-119; node/src/main.rs:837-841,1284-1287,1525-1540,2529 · Area: P2P / sync · Evidence: Confirmed by test (live; release predicate replicated)

Peer tip claims are clamped to our height + 1,000,000, but IBD is released only within +16; the table is monotonic-max and keyed to the relaying peer; directed Tips responses never update it, so the verified release path is dead. Unclamped values set `is_syncing`. The miner pauses, and after the 90 s stall valve it mines with `FINALITY_FROZEN = true`.

Failure scenario. One unauthenticated Version or PeerTip claim keeps every receiving node in IBD mode with mining paused or degraded.

Bitcoin Core. Sync state is driven by PoW-validated headers with `MinimumChainWork`; unverified peer claims do not change mining behaviour.

Recommendation. Trust only work-backed claims, attribute them to the author, and decay them.

High NET-7 — Gossip requests produce mesh-wide broadcast replies (O(N^2) amplification)

Location: node/src/main.rs:919-943,1023-1051; node/src/network/mod.rs:285-286,1152-1180,1596-1600 · Area: P2P · Evidence: Confirmed by test (100 unique message ids decode to the same request); amplification by reading

Any peer's gossip `GetHeaders`, `GetBlock` or `PeerRequest` makes every node broadcast a reply on gossip (up to 2,000 headers, a full block body or a `BlockNotFound` with a fresh nonce, or the whole known-peers table). `decode_wire_message` ignores trailing bytes, so requests can be made byte-unique and gossip de-duplication never stops them; failed directed pulls also re-publish on gossip.

Bitcoin Core. Requests are point-to-point; `getaddr` answers from a cached sample once per connection and address relay is rate-limited (0.1 addr/s token bucket).

Recommendation. Serve pulls only over directed request-response, reject trailing bytes, and rate-limit per author.

High OPS-1 — CI never builds or tests the consensus workspace; documented guards and fuzz targets do not exist

Location: .github/workflows/ci.yml; Cargo.toml:12; node/Cargo.toml:91; crates/gnb-crypto/Cargo.toml:68; crates/gnb-crypto/src/crypto/mod.rs:723,1552; node/src/mempool/mod.rs:376; node/fuzz/corpus/ · Area: CI / release engineering · Evidence: Confirmed by reading and by running the workspace locally (the suites do pass when run: 61 binaries, 1,040 executions, 0 failures)

CI runs only the Python reference and the standalone `rust/` crate, which is excluded from the workspace. The node, `gnb-crypto`, `coherence-core`, the consensus copy of the PoW in `crates/gnb-smh` and the wallet are never compiled or tested in CI; there is no clippy, cargo-deny, cargo-audit, fuzzing, sanitizer or Miri job. Comments cite `scripts/falcon-clean-guard.sh` and cargo-fuzz targets (`pow_verify`, `mempool_ops`) that do not exist; `node/fuzz` holds only seed corpora, replayed by `parser_corpus_floor.rs`.

Bitcoin Core. Bitcoin Core CI runs 26 configurations including ASan/UBSan/integer, MSan, TSan, valgrind, fuzzing with sanitizers, clang-tidy, IWYU, big-endian s390x and cross-OS builds (ci/test/00_setup_env_*.sh), plus a lint suite (test/lint).

Recommendation. Run `cargo test --workspace --release --locked`, clippy with `-D warnings`, cargo-deny and cargo-audit on every push; add cargo-fuzz targets for the wire decoders, PoW verify and mempool; add the Falcon guard or remove the claims.

High POW-3 — Near-free blocks on deep forks: absolute-schedule ASERT, a pow limit of ~2^256 and no anti-DoS work rule

Location: crates/gnb-crypto/src/core/mod.rs:150,185-203; node/src/main.rs:1103-1186,2024-2032,2182-2200,2546-2592; node/src/sync/parent_fetch.rs:94; node/src/consensus/mod.rs:1512-1521 · Area: Difficulty / DAG · Evidence: ASERT part confirmed by test; acceptance path by reading; mining-halt and recursion depth plausible

`POW_LIMIT_BITS = 0x2100ffff` is the largest decodable target (any nonce succeeds with probability 65535/65536, `work() = 1`). Expected bits depend only on the block's own parents, so a fork from any parent at least 24.27 h behind schedule needs one honest-difficulty proof and then yields unlimited children and siblings at the pow limit; forking from genesis six days after launch yields ~14,367 consecutive free heights. Blocks below finality are deliberately retained as fork-losers, and there is no minimum-chain-work or merge-depth rule. The backfill guard is bypassed by first sending a near-tip orphan that makes the node request (and therefore await) the deep fork blocks.

Failure scenario. Unbounded DAG/disk growth at about one evaluation per block, with multi-second insertions under the global DAG write lock. Because templates use every bodied tip as a parent, more than 256 tips would make every template violate `MAX_PARENTS` (plausible mining halt). `process_orphans` recurses once per accepted orphan, so a very deep chain risks stack exhaustion (plausible).

Bitcoin Core. Retargets are clamped (x4), `PermittedDifficultyTransition` bounds unverified headers, `MinimumChainWork`/`GetAntiDoSWorkThreshold` and presync reject low-work chains, and unrequested blocks need more work than the tip.

Recommendation. Add an anti-DoS blue-work threshold relative to the tip, reject parents below finality or merge depth, cap template parents (highest blue work first), make orphan processing iterative, and lower the pow limit to a target that costs real work.

High POW-4 — Orphan pool admission is gated only by self-declared bits: memory exhaustion and eviction of honest orphans

Location: node/src/main.rs:771,1144-1189; deploy/gnb-node.service (MemoryMax) · Area: P2P / orphan pool · Evidence: Confirmed by test (a 951,270-byte orphan with nonexistent parents admitted after one evaluation; ~19 GB per node at the 10,000-entry cap, versus MemoryMax=6G in the shipped unit) and by reading

Orphans (unknown parents) are admitted after `validate_structure`, so pow-limit bits make them free. Each entry stores the parsed block and its raw bytes (up to `MAX_BLOCK_SIZE` = 1 MB each, 10,000 entries), and eviction removes the lowest claimed height first, so attacker orphans with huge claimed heights are never evicted while honest ones are. `process_orphans` recurses once per accepted orphan on a 2 MiB worker stack, so a long attacker chain can also exhaust the stack (plausible).

Failure scenario. About 10 GB of upload pins about 20 GB of RAM (out-of-memory on typical hosts) and blocks honest orphan resolution during sync.

Bitcoin Core. Core keeps no orphan-block pool; it requests headers first and stores only blocks it asked for on chains with sufficient work.

Recommendation. Do not admit orphans on self-declared bits; cap the pool by bytes and per peer; evict by arrival and peer, never by claimed height.

High VAL-2 — No BIP30/BIP34 protection: duplicate coinbase txids overwrite UTXOs and rollback deletes the earlier block's coin

Location: crates/gnb-crypto/src/core/mod.rs:935-941; node/src/reorg.rs:272-277,559-567 · Area: Validation / UTXO · Evidence: Confirmed by test (audit_poc_validation.rs val2)

Nothing forces coinbases to be unique (no height commitment in the coinbase, no check against overwriting an unspent output). A miner can produce the same coinbase (same script, value and address within one halving epoch) in two blocks, giving one txid. `put_utxo` overwrites the coin, and when the second block is reorged out its rollback deletes the coin and its coinbase record on nodes that applied it, while nodes that never applied it keep them.

Failure scenario. A later spend of that coin is valid on one set of nodes and invalid on the other: a chain split triggered by a miner.

Bitcoin Core. BIP34 height in coinbase (`bad-cb-height`), BIP30 no-overwrite checks, and `DisconnectBlock` handling of overwritten coins (`DISCONNECT_UNCLEAN`) in src/validation.cpp.

Recommendation. Commit the height in the coinbase, refuse outputs that would overwrite an unspent coin, and make undo restore overwritten coins.

High VAL-3 — A block may spend its own coinbase (maturity bypass), while the reorg path forbids it: miner-triggerable split

Location: node/src/main.rs:2859-2882; crates/gnb-crypto/src/core/mod.rs:91-113; node/src/reorg.rs:654-658 · Area: Validation · Evidence: Confirmed by test (val3) and by reading (lead auditor re-verified)

The in-block prevout map includes `transactions[0]`, and `check_coinbase_maturity` only rejects inputs whose coinbase height is already stored, which is not yet the case for the block's own coinbase. A non-coinbase transaction can therefore spend the coinbase of the same block with zero confirmations. The reorg re-validation path rejects the same block (`immature same-block coinbase`).

Failure scenario. Nodes receiving the block as an extension apply it; nodes that held a sibling first store it as a fork-loser but can never reorg onto it. A miner can launder fresh subsidy immediately and split the network.

Bitcoin Core. `CheckTxInputs` enforces `COINBASE_MATURITY` for every spend, including within the block (src/consensus/tx_verify.cpp).

Recommendation. Exclude the coinbase from the in-block prevout map and use one shared rule function for acceptance and reorg.

High WAL-1 — Wallets sign with UTXO values supplied by the node/API and have no maximum-fee guard

Location: crates/gnb-crypto/src/wallet/mod.rs:376-436,1093-1174; crates/gnb-wallet-wasm/src/lib.rs:101-134; crates/gnb-crypto/src/wallet/client.rs:152-170; site/wallet/app.js:214-220 · Area: Wallet · Evidence: Confirmed by test (audit_poc_wallet.rs)

Because the sighash does not commit to spent amounts (CRY-3), a wallet that is told a wrong value for its own UTXO produces byte-identical signatures. Every builder (Rust `TxBuilder`, `Wallet::build_tx`, the WASM `pay`, both CLIs) trusts the values from RPC, none enforces a fee ceiling, and the web review screen shows the fee computed from the claimed values. `WalletClient::send` also accepts a node-chosen absolute fee.

Failure scenario. A compromised or spoofed API (see WAL-2) reports a 100 GNB coin as 1.0005 GNB; the user pays 1 GNB and the UI shows a 0.0005 GNB fee, while 99 GNB go to the miner.

Bitcoin Core. Core's wallet reads values from its own validated chain state, BIP143/BIP341 sighashes commit to amounts so a lie invalidates the signature, and `-maxtxfee` (default 0.1 BTC) is enforced (src/wallet/spend.cpp).

Recommendation. Enforce absolute and per-kB fee ceilings in every builder; verify input values (fetch and hash the previous transactions) before display; plan a sighash version that commits to spent amounts.

High WAL-2 — The Cloudflare Worker reaches the explorer origin over plain HTTP with the shared secret, and caches per origin headers

Location: worker/index.js:9,33,46-47,70 · Area: Web infrastructure · Evidence: Confirmed by reading

The origins are hard-coded `http://` endpoints on port 2086. The `x-gnb-key` secret, every UTXO and balance response, and every broadcast transaction cross the Internet unencrypted, and any origin response with `max-age>=1` is stored in the edge cache. No HSTS header is set.

Failure scenario. An on-path attacker between the edge and a VPS steals the API key, forges UTXO values (driving WAL-1 for every web-wallet user), injects markup into explorer data (WAL-3), and makes the forgery persist at the edge with a long `max-age`.

Bitcoin Core. Not applicable: Bitcoin Core ships no hosted web wallet or proxy; this is additional attack surface.

Recommendation. Use TLS or an authenticated tunnel to the origin, ignore origin cache headers in favour of fixed short TTLs, and add HSTS.

Medium CRY-1 — Consensus accepts third-party witness malleation and nothing commits to script_sig bytes

Location: crates/gnb-crypto/src/crypto/mod.rs:229-234,280-288,752-760; crates/gnb-crypto/src/core/mod.rs:626-629,678-710,876-895; node/src/main.rs:2726-2744 · Area: Signatures / block format · Evidence: Confirmed by test (audit_poc_crypto.rs); also reported by the validation and PoW reviewers

Non-coinbase txids exclude `script_sig` (anti-malleability, like a segwit txid), and the Merkle root is built from txids, but there is no witness commitment. On top of that, the consensus verifier is lenient: a third party can strip or add the 4-byte suite envelope, zero-pad the compact Falcon signature to the 1,280-byte padded form, and append arbitrary bytes after the public key (up to the 10,000-byte script cap). The repository already contains a strict `verify_enveloped_canonical`, but consensus does not use it.

Failure scenario. A relay pads the witnesses of a valid near-full block. Block id, Merkle root and txids are unchanged, but `size_bytes()` grows (+1,659 bytes per input in the PoC), so `MAX_BLOCK_SIZE` and the size-based minimum-fee rule give a different verdict for the same block id. Nodes can also store and serve different bytes for one id. No invalid-block-id cache exists, and gossip ids hash the full payload, so permanent rejection of the honest block was not observed.

Bitcoin Core. Strict DER, low-S and NULLFAIL rules (src/script/interpreter.cpp) remove ECDSA malleability; segwit commits witnesses through the coinbase witness commitment (`BlockWitnessMerkleRoot`, src/consensus/merkle.cpp) checked by `CheckWitnessMalleation` (src/validation.cpp), and `ComputeMerkleRoot` reports mutation.

Recommendation. In the next hard fork require an exact-length `script_sig`, a mandatory envelope and canonical Falcon encoding (`verify_enveloped_canonical`), and commit to witness data (header or coinbase witness root). Keep never caching invalidity by block id.

Medium DAG-10 — accept_block is not serialized across ingress paths

Location: node/src/main.rs:1194,1715,1893,2061,2077,2096,2154,2230 · Area: Consensus / concurrency · Evidence: Plausible (narrow race window, traced by reading)

The network processor, the miner and RPC `submitblock` call `accept_block` concurrently without a global lock; tip snapshot, transaction validation, DAG insertion, new-tip read and UTXO application are separate critical sections. Interleavings can apply two sibling extensions (UTXO set T+a+b while the selected chain is T->b) or accept a child validated before its parent was applied. Because both siblings can be applied, a conflicting spend can be applied twice, inflating supply, and the undo record then misses the input.

Bitcoin Core. `m_chainstate_mutex` plus `cs_main`, with `processnewblock_signals_ordering` driving `ProcessNewBlock` from 10 threads in src/test/validation_block_tests.cpp.

Recommendation. Hold one acceptance mutex across the whole pipeline and add a multi-threaded test.

Medium DAG-4 — The live (fast) GHOSTDAG coloring violates the k-cluster rule

Location: node/src/consensus/mod.rs:1253-1254,1278-1364,1396-1423 · Area: Consensus / GhostDAG · Evidence: Confirmed by test (k=10: a side block under a 30-block chain is colored blue; random differential vs an exact PHANTOM reference: 443 of 920 blocks get a different blue set)

The selected parent is stripped from `mergeset_blues`, but the ported bounded chain walk assumes Kaspa's layout where it is element 0, so selected-chain blocks below the immediate selected parent are never counted in a candidate's anticone; historical anticone sizes are reset to 0. The legacy path has the same omission, so the Legacy-vs-Fast differential test cannot detect it. Effect: inflated blue scores, which feed tie-breaks, IBD ordering and RPC.

Recommendation. Test against an exact GHOSTDAG reference (and Kaspa vectors) and fix the anticone walk.

Medium NET-10 — Unbounded frontier state and fan-out from unsolicited frames

Location: node/src/sync/frontier.rs:88-103; node/src/main.rs:898-913,962-978 · Area: P2P / sync · Evidence: Confirmed by test (2,000 Tips frames -> 512,000 in-flight entries and 512,000 GetBlocks)

Tips frames are bounded individually but not in total, overheard Headers also trigger fetches, and the abandoned set is never pruned; IBD stays latched.

Recommendation. Bound frontier state globally and per peer and only act on solicited responses.

Medium NET-11 — Weak peer exchange and eclipse resistance

Location: node/src/network/mod.rs:841-847,1181-1248,1501-1513; node/src/network/pex_validator.rs:296-307; node/src/network/sync_rr.rs:157; node/src/main.rs:1294 · Area: P2P / peer management · Evidence: Confirmed by reading

Unsolicited relayed PeerExchange messages trigger immediate dials to up to 20 attacker-chosen addresses per message, rate-limited per relayer rather than per author (a reflection and port-scan amplifier). The known-peers table uses arbitrary eviction with no netgroup buckets, tried/new tables, anchors or feeler connections; inbound slots are first-come; three free identities confirm the external address; directed GetPeers answers are discarded.

Bitcoin Core. addrman with keyed netgroup buckets and new/tried tables (src/addrman_impl.h), inbound eviction protecting diverse peers (`AttemptToEvictConnection`), anchors, feelers and block-relay-only connections (src/net.cpp).

Recommendation. Adopt an addrman-like design and dial only from the table, never directly from received messages.

Medium NET-12 — The public explorer chain allows unbounded full-chain scans

Location: worker/index.js; deploy/explorer-api/gnb_explorer_api.py; node/src/rpc/auth.rs:150,417; node/src/rpc/mod.rs:189-202,620-652,904-918; node/src/storage/mod.rs:297-315 · Area: RPC / explorer · Evidence: Confirmed by reading

Public requests flow through the Worker (no rate limit) and the explorer API (which allows `getdifficultyhistory` with 500 parameter values, `getblocktimepercentiles` and `getutxos`) to the node RPC over loopback, which is exempt from rate limiting. Two of these methods iterate every block body and are not classed as heavy reads; aborting the blocking task does not stop a running scan, so the 256-request concurrency cap does not bound work; `getutxos` loads all of an address's UTXOs before applying the limit.

Recommendation. Class scanning methods as heavy, precompute their results, add cancellation, and rate-limit by original client.

Medium NET-8 — Rate-limit and penalty maps never evict

Location: node/src/network/sync_rr.rs:427-483; node/src/network/mod.rs:394-452 · Area: P2P · Evidence: Confirmed by test

`SyncRequestLimiter` fails closed with no eviction: after 1,024 free identities every new peer is refused forever (and, through NET-5, newcomers then penalise the victim). `WirePenaltyTracker` saturates at 1,024 identities and keeps later offenders at -100 forever.

Recommendation. Use bounded LRU maps with time-based decay.

Medium NET-9 — A single attacker can capture and stall all IBD pulls

Location: node/src/network/sync_rr.rs:513-537,568-577; node/src/sync/frontier.rs:117-130 · Area: P2P / sync · Evidence: Confirmed by test

`select_pull_peer` prefers peers that declare themselves archival in the identify agent string, then the highest announced score, with no stall detection or rotation; two `NotFound` replies abandon a tip.

Bitcoin Core. Parallel download from several peers with stalling detection and disconnection (`BLOCK_STALLING_TIMEOUT`, `CHAIN_SYNC_TIMEOUT`).

Recommendation. Rotate peers, detect stalls, and ignore self-declared capabilities for selection.

Medium OPS-2 — Dependency policy is stale and not enforced; reachable advisories are ignored

Location: deny.toml; Cargo.lock · Area: Supply chain · Evidence: Confirmed by running cargo-deny 0.20.2 and cargo-audit 0.22.2

`cargo deny check` fails the project's own license policy (the three first-party AGPL crates are not allowed) and reports 5 unmatched and 2 unnecessary skips and 15 license entries never encountered (inherited from Bloch, including SP1 dependencies that are no longer present). cargo-audit reports 11 vulnerabilities in `Cargo.lock`: quinn-proto (5; in the lockfile but not compiled for any target), rustls 0.23.45 (4; reachable through reqwest and libp2p-websocket, fixed in 0.23.46) and hickory-proto 0.25.2 (2; DoS, reachable through libp2p-dns/mdns; ignored in deny.toml without rationale). The PQClean-based crates are flagged unmaintained (upstream archived).

Recommendation. Repair deny.toml, enforce it in CI, upgrade rustls and hickory, prune the lockfile, and plan a maintained PQ backend.

Medium POW-5 — Consensus difficulty code is weakly tested: feature unification disables the real ASERT path in workspace tests, 37 mutants survive, no external oracle

Location: crates/gnb-crypto/Cargo.toml (test-pow-limit); node/Cargo.toml (dev-dependencies); crates/gnb-crypto/src/core/mod.rs:1297; crates/gnb-smh/src/difficulty.rs:249,255; reference/gnb_smh.py · Area: Difficulty / tests · Evidence: Confirmed by reading, by build fingerprints, by mutation testing (cargo-mutants 27.1.0) and by execution: the PoW proof-of-concept tests in gnb-crypto pass 5/5 when the crate is tested alone and 2 of them fail when it is built in the same cargo invocation as the node, because the node dev-dependency feature `test-pow-limit` is unified into gnb-crypto

`cargo test --workspace` unifies the node's dev-dependency feature `test-pow-limit` into `gnb-crypto`, so `expected_bits_at_schedule_is_genesis_bits` is ignored and every node test uses a constant expected target. The normative Python reference defines no difficulty rule. Mutation testing of `crates/gnb-smh` left 37 of 213 viable mutants alive (82.6% score); besides equivalent mutants they include a schedule off-by-one (`height - anchor - 1` -> `+ 1`), the strict PoW comparison (`<` -> `<=`), `verify`, `hash` and `work`. GnB's code comment calls its floor-division exponent `like aserti3-2d`; under C++-style truncation the reviewer's model disagrees on 42% of sampled compact results, while the audit harness (floor division, BCHN fixed point) agrees bit-for-bit, so GnB's rule is only pinned by its own code.

Bitcoin Core. src/test/pow_tests.cpp, arith_uint256_tests.cpp and headers_sync_chainwork_tests.cpp, plus fuzz targets `pow`, `pow_transition` and `headerssync`, run in CI on every configuration.

Recommendation. Test `gnb-crypto` without `test-pow-limit` in a separate CI job; publish ASERT and chain-work vectors in the Python reference; adopt the audit harness (audit/consistency) in CI; fix the comment to state the exact rounding rule.

Medium WAL-3 — Explorer and landing pages have no CSP and render API data through innerHTML without escaping

Location: worker/index.js:88-96; wrangler.toml:17; site/explorer.html:219,334,351,357,399,415-420,430,489; site/wallet/app.js:181 · Area: Web wallet / explorer · Evidence: Wallet sink confirmed by test; explorer by reading

The CSP is applied only under `/wallet`. The explorer interpolates API fields (links, counts, percentiles, ranks) into `innerHTML`, and the wallet inserts `utxo_count` raw. In the wallet, the strict CSP blocks script execution but HTML injection and meta-refresh remain; on the explorer, a forged API response (WAL-2) yields script execution on the same origin, enough for a convincing fake wallet. On-chain strings (coinbase text, addresses) are escaped or canonical.

Recommendation. Escape every interpolation or use `textContent`, validate numeric and hex types, and apply a strict CSP site-wide.

Medium WAL-4 — Explorer API is exhaustible by idle connections and is not firewalled to the proxy

Location: deploy/explorer-api/gnb_explorer_api.py:34,245; deploy/explorer-api/cloudflare-ips-v4.txt; node/src/rpc/auth.rs:295 · Area: Explorer API · Evidence: Confirmed by test (50 idle connections held 50 threads)

`ThreadingHTTPServer` binds 0.0.0.0:2086 with unbounded threads and no socket timeout, so a slowloris-style client exhausts it before the key check. The Cloudflare IP list is shipped but never applied. Its `sendrawtransaction` calls reach the node from loopback, which is exempt from the node's RPC rate limiter.

Recommendation. Set handler timeouts and a concurrency cap, bind to loopback or a tunnel, apply the IP allow-list, and rate-limit by original client.

Low AUD-1 — ASERT saturates its intermediate product before a negative shift: up to ~2x too hard for anchors near the pow limit

Location: crates/gnb-smh/src/difficulty.rs:270-283,306-330; audit/consistency/tests/asert_differential.rs (high_anchor_saturation_diverges_from_aserti3_2d) · Area: Difficulty (latent) · Evidence: Confirmed by the audit harness (differential against the aserti3-2d fixed-point step)

`mul_u64_shr16` clamps `target * factor >> 16` to 2^256 - 1 before the right shift by a negative exponent. For anchor targets of roughly 2^255 and above, the result is harder than the exact arithmetic (for anchor `0x2100a34e`, eight half-lives behind schedule, GnB yields `0x1f7fffff` where the exact rule yields `0x2000a349`). BCHN avoids this by requiring 32 spare leading bits in the pow limit. The deployed genesis anchor (`0x1f0e9036`, about 2^244) has 11 bits of headroom, so current consensus is unaffected; the defect becomes live only if ASERT is re-anchored near the pow limit.

Recommendation. Compute the product in 288+ bits and apply the net shift once (as aserti3-2d does), or require headroom between the anchor and 2^256.

Low AUD-2 — The vendored crates' own guard tests cannot be run in place

Location: crates/pqcrypto-internals/tests/vendor_pin.rs; crates/libp2p-yamux/tests/lockfile_guard.rs; Cargo.toml:5-12 · Area: Supply chain / tests · Evidence: Confirmed by running cargo (`current package believes it's in a workspace when it's not`)

`crates/pqcrypto-internals` and `crates/libp2p-yamux` are patched path dependencies but neither workspace members nor excluded, so `cargo test` refuses to run inside them and `--workspace` does not include them. The PQClean source-pin and yamux lockfile guards described in the manifests therefore never run. Copied into an isolated workspace they pass (vendor_pin 2/2, lockfile_guard 2/2, plus 17 and 2 unit tests).

Recommendation. Add both crates to `workspace.members` (or `exclude` with their own `[workspace]`) and run them in CI.

Low COH-2 — Shielded state is mutated before the block's disposition is known and never rolled back

Location: node/src/main.rs:2093,2099-2340; node/src/reorg.rs:841-871; node/src/main.rs:715-716 · Area: Shielded pool (inactive) · Evidence: Confirmed by reading

`apply_block_self` runs before `add_block` and before the finality and refused-reorg exits, fork-loser blocks also mutate it, reorgs never disconnect it, and the state lives only in memory. Inert today (no shielded transaction is accepted); if enabled it would make state depend on arrival order and forget nullifiers across restarts. `apply_block` is also quadratic (full clones and O(n) inserts).

Recommendation. Apply shielded state only for blocks that become part of the selected chain, include it in undo data, and persist it.

Low CRY-2 — ML-DSA-only suite 0x0002 is live without activation and hybrid signatures are separable

Location: crates/gnb-crypto/src/crypto/mod.rs:270-275,365,373,457-470 · Area: Signatures · Evidence: Confirmed by test

`crypto::verify` dispatches the single-algorithm suite with no height gate, `is_hybrid_public_key` has no caller, and the per-component messages do not bind the suite id, so stripping the Falcon half and relabelling a hybrid signature as 0x0002 yields a valid ML-DSA-only signature for the 0x0002-wrapped key. Addresses hash the enveloped key, so existing hybrid outputs cannot be downgraded, but anyone can lock funds to single-algorithm outputs today.

Recommendation. Gate suite 0x0002 by height or policy and bind suite id and a domain tag into each component signature (composite-signature practice).

Low CRY-3 — Sighash preimage is defined by a serialization library and does not commit to spent amounts

Location: crates/gnb-crypto/src/core/mod.rs:645-676 · Area: Signatures · Evidence: Confirmed by test (independent encoder matches)

SIGHASH_ALL covers version, all outpoints and sequences, the signed input index (via a marker), all outputs, locktime, a 16-byte domain, a version byte and the chain id, which is sound against replay. The body bytes are whatever `bincode` 2 serde emits, so consensus depends on a library's encoding rather than a specification, and there is no golden vector in the repository. Spent values and scripts are bound only through the outpoint, which matters for offline signers (see WAL-1). An out-of-range input index silently omits the marker.

Bitcoin Core. BIP143 (`SignatureHash`) commits the spent amount; BIP341 (`SignatureHashSchnorr`) commits all spent amounts and scripts under a tagged hash; `src/test/data/sighash.json` pins 500 vectors.

Recommendation. Specify the preimage byte-by-byte, add amount and script commitments in a new sighash version, assert the index is in range, and pin golden vectors.

Low CRY-4 — Secret keys, mnemonics and passwords are not reliably zeroized

Location: crates/gnb-crypto/src/crypto/mod.rs:86-97,203,746; crates/gnb-crypto/src/hd_wallet/mod.rs:142,202-204; node/src/bin/gnb-cli.rs:652-658; crates/gnb-wallet-wasm/src/lib.rs:52,72 · Area: Keys / memory hygiene · Evidence: Confirmed by test and by reading

pqcrypto `SecretKey` types are `Copy` without `Drop`; `generate_keypair` returns the secret as a plain `Vec`; `bip39` is built without its `zeroize` feature and the wallet keeps the `Mnemonic` for its lifetime; the CLI reads passwords and mnemonics into plain `String`s; the WASM core keeps the password in a `serde_json::Value`.

Bitcoin Core. Wallet secrets use `secure_allocator` (locked, wiped pages) and `memory_cleanse` (src/support/allocators/secure.h, src/wallet/crypter.h).

Recommendation. Wrap all secret buffers in `Zeroizing`, enable `bip39/zeroize`, and avoid copying `SecretKey` values.

Low CRY-5 — Falcon signer return code is discarded and signatures are not verified after signing

Location: crates/gnb-crypto/src/crypto/mod.rs:203,746; pqcrypto-falcon falcon1024.rs:296-309 (dependency) · Area: Signatures · Evidence: Confirmed by test

With a corrupted Falcon secret key `crypto::sign` returns `Ok` with an empty Falcon half; the failure only surfaces when the transaction is rejected. The behaviour is fail-closed but silent.

Bitcoin Core. `CKey::Sign` verifies every signature before returning it (src/key.cpp).

Recommendation. Check the output length and verify each signature before returning it.

Low CRY-7 — Address checksum and the default keystore format do not bind the network

Location: crates/gnb-crypto/src/address.rs:109-116; crates/gnb-crypto/src/wallet/mod.rs:996-999,1044,1071,1161-1165; node/src/main.rs:186 · Area: Addresses / keystores · Evidence: Confirmed by test

The 4-byte checksum covers only the key hash, so `gnb1q` <-> `gnb1t` swaps pass, and the node's `address_to_script_pubkey` ignores the network. The keystore v2 written by every front end encrypts with empty AAD and ignores its `network` field, so editing the prefix in the file loads the key for the other network. The v1 keyfile binds network and public key in AAD and rejects the same edit. The wallet already rejects cross-network recipients (prior finding CR-05).

Recommendation. Use a checksum over prefix and payload (e.g. bech32m-style) in a new address version and bind network and public key in the v2 keystore AAD.

Low DAG-5 — Restart re-marks interior blocks as tips

Location: node/src/consensus/mod.rs:895,953 · Area: Consensus / persistence · Evidence: Confirmed by test (tips go from 1 to 2 after reload)

`load_persisted*` inserts blocks in blue-score order, which is not topological for non-selected parents, so after a restart such parents become tips again (redundant template parents; tip sets differ between nodes).

Recommendation. Reload in topological order and recompute tips.

Low DAG-8 — DAG height and header height disagree

Location: node/src/consensus/mod.rs:306-308,1041; node/src/main.rs:2012 · Area: Consensus · Evidence: Confirmed by test (required header height 6, DAG height 2)

`GhostdagData.height = selected_parent.height + 1`, but the header must declare `max(parent heights) + 1`. They diverge whenever the selected parent is not the highest parent, which also invalidates the height shortcut in the legacy `is_ancestor`.

Recommendation. Use one definition of height throughout.

Low MEM-6 — Policy gaps: no rolling minimum fee, no replacement policy, no package/orphan handling; fee estimate defaults below the consensus minimum

Location: node/src/mempool/mod.rs:231-259,338-344; node/src/rpc/mod.rs:1031-1039; crates/gnb-crypto/src/core/tokenomics.rs:85 · Area: Mempool · Evidence: Confirmed by reading

Eviction does not raise an incremental relay fee, first-seen is final, child transactions of unconfirmed parents are not supported, and `estimatefee` returns a median absolute fee that defaults to 1,000 sat while the consensus minimum is 10,000 sat.

Recommendation. Add a rolling minimum fee and an explicit replacement rule; return fee rates bounded below by the consensus minimum.

Low MEM-7 — Reorg re-injection skips maturity and minimum-fee re-checks

Location: node/src/reorg.rs:695-734 · Area: Mempool / reorg · Evidence: Confirmed by reading

Transactions from disconnected blocks re-enter the pool without the maturity and fee checks applied at normal admission, so a reorg to a lower height can re-admit immature spends that then poison templates.

Recommendation. Route re-injected transactions through the same admission function as gossip.

Low NET-13 — RPC method strings become unbounded Prometheus labels and log lines

Location: node/src/rpc/mod.rs:519-528,655-676 · Area: RPC / metrics · Evidence: Confirmed by test (2,000 series created)

The method name (up to the 1 MiB body) is used as a permanent metrics label and logged at info level on every path, including forbidden requests evaluated before the rate limiter.

Recommendation. Map unknown methods to a single label and truncate logged values.

Low NET-14 — The node exits if the metrics or RPC server fails to bind

Location: node/src/metrics/mod.rs:422-448; node/src/rpc/mod.rs:319-324,391-394; node/src/main.rs:1791-1818 · Area: Node lifecycle · Evidence: Confirmed by reading

A bad address or failed bind returns from the server task, which ends the top-level `select!` and stops the node (for example `--metrics-addr ::1` fails to parse).

Recommendation. Validate addresses at start-up and treat auxiliary server failures as non-fatal.

Low NET-15 — Explorer calls hold the DAG read lock across storage reads

Location: node/src/rpc/explorer.rs:29-64,178-214 · Area: RPC / explorer · Evidence: Confirmed by reading

Explorer handlers keep `dag.read()` across per-transaction and per-input RocksDB reads and rebuild an O(chain) cache, which can starve the `accept_block` writer.

Recommendation. Copy the needed DAG data and release the lock before storage access.

Low NET-16 — Secret file and authentication hygiene

Location: node/src/network/mod.rs:1808,1823-1827; node/src/main.rs:269-289; node/src/rpc/auth.rs:302,426 · Area: Secrets / RPC auth · Evidence: Confirmed by reading

`p2p_identity.bin` is written with default umask permissions; `known_peers` is written non-atomically; API key file permissions are not checked; the constant-time comparison short-circuits on length (leaking key length); failed authentication has no delay; `gnb-cli` sends the key over plain HTTP.

Bitcoin Core. Cookie authentication with restricted file permissions, salted HMAC `rpcauth` with `TimingResistantEqual`, and a 250 ms delay after failed authentication (src/httprpc.cpp).

Recommendation. Write secrets 0600 atomically, compare in constant time over fixed-length digests, and delay failed attempts.

Low OPS-3 — Systemd units lack most sandboxing directives; installer quoting issues

Location: deploy/gnb-node.service; deploy/explorer-api/gnb-explorer-api.service; deploy/install-host.sh; deploy/gnb-launch · Area: Deployment · Evidence: Confirmed by reading

The node unit sets `NoNewPrivileges`, `ProtectSystem=strict`, `ProtectHome`, `PrivateTmp` and `MemoryMax`, but not `PrivateDevices`, `ProtectKernel*`, `RestrictAddressFamilies`, `RestrictNamespaces`, `SystemCallFilter`, `CapabilityBoundingSet=` or `MemoryDenyWriteExecute`; the explorer API runs as the same user. `install-host.sh` writes operator input into an env file through an unquoted heredoc and `gnb-launch` expands `GNB_PEERS` unquoted. RPC and metrics bind to loopback (positive).

Recommendation. Add the missing directives, run the API as a separate user, and quote operator input.

Low POW-6 — work() deviates from the normative definition floor(2^256 / target)

Location: crates/gnb-smh/src/difficulty.rs:165-184; reference/gnb_smh.py:109-113 · Area: Difficulty · Evidence: Confirmed by test (100,497 of 198,463 sampled targets differ; harness bounds the error)

The Rust code computes floor((2^128 - 1) / ((t >> 128) + 1)); the Python specification says floor(2^256 / t), and Bitcoin Core uses floor(2^256 / (t + 1)). The audit harness shows GnB never overstates work and understates it by less than 2^-64 relative for every target GnB can reach, so Rust nodes agree with each other; an implementation written from the specification would compute different blue work.

Recommendation. Make the specification and the code identical (preferably exact 256-bit arithmetic) and pin vectors.

Low POW-7 — The difficulty input is documented as the selected parent but is the latest timestamp over all parents; two notions of height coexist

Location: node/src/pow/mod.rs:6-7; crates/gnb-crypto/src/core/mod.rs:140-142,185-188,966; node/src/consensus/mod.rs:1041 · Area: Difficulty / documentation · Evidence: Confirmed by reading and by test

Consensus uses `max(parent timestamps)` and `max(parent heights) + 1`, while the DAG's own height is selected-parent height + 1. The result is deterministic but couples block validity to GhostDAG ordering in a way the documentation does not describe.

Recommendation. Document the exact rule in docs/CONSENSUS.md and add multi-parent difficulty tests under real ASERT.

Low POW-8 — Wire metadata (height, blue_score) is not committed by the block id; blue_score is never validated

Location: crates/gnb-crypto/src/core/mod.rs:311-320,344-358,805-811; node/src/main.rs:1030,1188,1911 · Area: Block format · Evidence: Confirmed by test

`blue_score` travels with the block, is persisted and re-served without validation, and a later copy with different metadata overwrites an honest orphan with the same id. No invalid-id cache exists, so nothing is poisoned permanently.

Recommendation. Stop transmitting derived metadata, or validate it against the locally computed values before storing or serving it.

Low VAL-10 — Merkle tree has no leaf/inner-node domain separation and no 64-byte transaction rule

Location: crates/gnb-crypto/src/core/mod.rs:678-696 · Area: Block format · Evidence: Confirmed by reading

A 64-byte stripped transaction (one input, one output, 5-byte script) can be constructed, so an inner node can be presented as a leaf in a Merkle proof. Full nodes are protected by the duplicate-transaction rule, coinbase-first rule and absence of an invalid cache; the impact is limited to light clients and proofs.

Bitcoin Core. `IsBlockMutated` rejects 64-byte transactions (src/validation.cpp); BIP-proposals separate leaf and node hashing.

Recommendation. Use tagged hashes for leaves and inner nodes in the next block version, or forbid 64-byte transactions.

Low VAL-12 — Script size limits live only in the wire parser; block size uses a different encoding than fees

Location: crates/gnb-crypto/src/core/mod.rs:589,604,1012,1021 · Area: Validation · Evidence: Confirmed by test (in-process block with a 12.6 kB script_sig is accepted, then `get_block` fails)

The 10,000-byte script limits are enforced only when parsing wire bytes, so a block constructed in-process (RPC, stratum, miner) can carry larger scripts, be accepted and persisted, and then fail to load, breaking rollback and serving. `MAX_BLOCK_SIZE` is measured in bincode while fees use the wire size. Not reachable from current network ingress.

Recommendation. Enforce all size rules in the consensus validation functions on the canonical encoding.

Low WAL-5 — WASM wallet core is loaded without an integrity check and the committed binary is not reproducibly tied to source

Location: site/wallet/core.js:6-12; site/wallet/app.js:47; scripts/build-wallet-wasm.sh · Area: Web wallet supply chain · Evidence: Confirmed by reading

`loadCore` supports a SHA-256 check but is called without one; scripts carry no SRI; `site/wallet/gnb_wallet.wasm` is a committed artifact whose hash is printed by the build script but pinned nowhere.

Recommendation. Build the WASM reproducibly in CI, publish and pin its hash, and pass it to `loadCore`.

Low WAL-6 — Coin selection and fee handling: smallest-first greedy selection, size-blind fees, silent dust-to-fee

Location: crates/gnb-crypto/src/wallet/mod.rs:422,1118-1150; crates/gnb-crypto/src/wallet/cli.rs:266; crates/gnb-crypto/src/wallet/client.rs:118; crates/gnb-wallet-wasm/src/lib.rs:128-131 · Area: Wallet · Evidence: Confirmed by test

Paying 1 GNB selected six 0.2 GNB inputs (~50 kB) over one 10 GNB coin. Some send paths use fixed fees, one fallback (10,000 sat) is below the consensus minimum for any one-input transaction, change of 546 sat or less is silently added to the fee while the WASM reports the requested fee, and the dust comparison differs between two code paths (`>=` vs `>`).

Recommendation. Use size-aware fees from the consensus formula, a waste-based selector, and report the actual fee.

Info ATT-1 — Remote attestation is not used for consensus or peer trust and fails closed

Location: node/src/attestation/mod.rs:294-306; node/src/attestation/sev_snp.rs:63-77; node/src/attestation/mobile.rs:29-41 · Area: Attestation (inactive) · Evidence: Confirmed by reading

Only the `getattestation` RPC exposes it, returning an honest no-TEE report; the SEV-SNP provider returns `None`, the mobile verifiers reject, and `verify()` fails closed without a quote verifier.

Recommendation. No action needed; keep it out of consensus.

Info COH-1 — Shielded spend statement does not bind spending authority (known CR-01); unreachable today

Location: crates/coherence-core/src/lib.rs:425-483; crates/coherence-core/tests/known_unsafe_v1_authorization.rs; node/src/coherence/verifier.rs:40; node/src/main.rs:2093 · Area: Shielded pool (inactive) · Evidence: Confirmed by test (8 spends of one 1,000-value note)

`check_spend` does not bind the nullifier key to the note's `pk_d`, so anyone with the note plaintext can spend it and one leaf yields unlimited fresh nullifiers. Today the only compiled verifier is `RejectAll`, the `sp1-verify` feature does not exist, the verification-key pin is empty, and any block containing a shielded transaction is rejected, so there is no inflation path. It would be Critical if the shielded pool were enabled as-is.

Recommendation. Keep the pool disabled until the statement binds authority and the state handling in COH-2 is fixed; remove the dormant code from consensus builds.

Info CRY-6 — Unused cryptographic dependencies are compiled (blst, secp256k1)

Location: node/Cargo.toml:59,146 · Area: Dependencies · Evidence: Confirmed by reading (no .rs reference; C/asm built, not linked)

`blst` and `secp256k1` (with `rand-std`) are declared but referenced by no source file. Their C/assembly is compiled on every build (supply-chain surface) although no symbol reaches the binary.

Recommendation. Remove both dependencies.

Info CRY-8 — Minor: 32-bit overflow in script parsing, message-signing domain without chain id, stale comments

Location: crates/gnb-crypto/src/core/mod.rs:702; crates/gnb-crypto/src/crypto/mod.rs:172,1372-1376; node/tests/kat_falcon1024.rs:9-14 · Area: Signatures · Evidence: Confirmed by reading

`4 + sig_len + 4` can overflow (and panic, overflow checks are on) on 32-bit targets; the only wasm32 caller is a test. Signed messages use `BLOCH-SIGNED-MESSAGE-v1` with no chain id, so message signatures carry over between Bloch, GnB mainnet and testnet, which share keys. Several comments still describe Falcon as floating-point and non-reproducible although the clean variant is pinned by a KAT.

Recommendation. Use checked arithmetic, add the chain id to the message domain, and refresh the comments.

Info DAG-12 — Minor GhostDAG observations

Location: node/src/consensus/mod.rs:1548,1564; node/src/consensus/reachability.rs:18; node/src/storage/mod.rs:223; node/src/main.rs:933 · Area: Consensus · Evidence: Confirmed by reading

`is_final` compares blue score with a height constant; `is_blue` uses the bounded legacy walk; doc comments describe disabled gates that are live; `put_block` overwrites the height->hash index with fork-losers, so `getblockhash` and pruning can act on the wrong block; duplicate parents are allowed; the gossip `GetHeaders` limit is not clamped (the directed path is). No attacker-reachable panic was found in the DAG code itself.

Recommendation. Clean up as part of the consensus fixes.

Info LED-1 — Ledger v1 sparse Merkle tree: no soundness defect found; witnesses uncommitted and coinbase uniqueness not required

Location: rust/src/ledger.rs:134-264,416-435,626-631,798-812; reference/gnb_ledger.py · Area: Ledger v1 prototype · Evidence: Confirmed by reading and by the vector suites

Leaf and node hashing are domain-separated, depth is fixed, the default-subtree shortcut equals the computed node, proofs are canonical and decoders reject trailing bytes; Rust and Python agree on every rule and error code. Open items for the prototype (not wired into the node): an uncommitted witness lets a bad final proof cost up to ~8.4M Keccak-f calls per attempt, identical coinbases at one height make the second block permanently invalid, and authorization is test-only.

Recommendation. Commit the witness bundle, require coinbase uniqueness, and add a randomized Rust-vs-Python differential before wiring it in.

Info NET-17 — Dead or diverging networking code

Location: node/src/dandelion.rs:74,108-121; node/src/transport/mod.rs; node/src/transport/upgrade.rs; node/src/network/mod.rs:577-587; node/tests/wire_decoder_fuzz.rs:8 · Area: P2P · Evidence: Confirmed by reading

Dandelion++ has no callers (and deviates from the protocol: per-transaction coin, single successor, fixed embargo, unbounded map), so transactions are gossiped directly from their origin. The ML-DSA handshake in `transport/mod.rs` is unused; the live transport uses Ed25519 plus round-3 Kyber768 rather than ML-KEM. Some comments and test headers reference fuzz targets and backends that do not exist.

Recommendation. Remove dead code or wire it in, and correct the documentation.

Info POW-9 — Minor PoW observations

Location: crates/gnb-crypto/src/core/mod.rs:330-335,785-932; crates/gnb-smh/src/lib.rs:188-202; crates/gnb-smh/src/difficulty.rs:91-128; rust/Cargo.toml · Area: PoW · Evidence: Confirmed by reading and by test

Pow-limit blocks can churn the 16,384-entry PoW cache; each verifying thread keeps a 64 MiB thread-local scratchpad for its lifetime; non-canonical compact encodings decode (17.7 M of 268 M accepted values in the exhaustive sweep), which is safe only because consensus compares `bits` to `expected_bits` as a u32; `validate_pow` has no explicit `target <= pow_limit` check (unreachable today because the largest decodable target equals the limit); `rust/Cargo.toml` lacks `overflow-checks`; `dag_hash()` is dead code; mainnet and testnet share the genesis and ASERT anchor.

Recommendation. Add the explicit pow-limit check, reject non-canonical bits at decode, and remove dead code.

Info VAL-11 — No MoneyRange check and no supply-sum audit; the 10 M GNB cap is implied, not enforced

Location: crates/gnb-crypto/src/core/tokenomics.rs:27,52-62; crates/gnb-crypto/src/core/mod.rs:948-963,982-991 · Area: Monetary policy · Evidence: Confirmed by reading; schedule verified by the audit harness (899,999,851,392,253 sat of 900,000,000,000,000 mined)

The cap follows from subsidy <= INITIAL >> halvings, coinbase <= subsidy + fees and value conservation; nothing compares values with `MAX_SUPPLY_SAT` at validation time and there is no UTXO-set total audit. Zero-value outputs to arbitrary scripts are allowed, which can bloat the UTXO set. All sums use checked arithmetic, so CVE-2010-5139-style wrap-around is not possible.

Bitcoin Core. `MoneyRange` on every output, input sum and fee (src/consensus/tx_check.cpp, tx_verify.cpp); `gettxoutsetinfo` and the `utxo_total_supply` fuzz target audit total supply.

Recommendation. Add MoneyRange-style checks as defence in depth and a supply invariant test.

Info WAL-7 — Wallet design notes: no mnemonic backup for default wallets, network chosen by file name, HTTP-only RPC client

Location: crates/gnb-wallet-wasm/src/lib.rs:64-69; crates/gnb-crypto/src/wallet/cli.rs:156; node/src/bin/gnb-cli.rs:279,431,631; crates/gnb-crypto/src/wallet/http_rpc.rs:1-23 · Area: Wallet UX / transport · Evidence: Confirmed by reading

Default wallets are random keys whose only backup is the encrypted file; `gnb-cli` picks the network from whether the file name contains `testnet`; the RPC client is HTTP-only and sends the API key in clear to any `--rpc-host`; neither CLI asks for confirmation before broadcasting.

Recommendation. Offer mnemonic-backed creation by default, record the network in the file, support TLS, and confirm before broadcast.

7. Component comparison with Bitcoin Core

Each table compares a GnB component with its Bitcoin Core counterpart on design, verified-correct behaviour, weaknesses found, tests and the reviewers' scores (consensus correctness / defensive coding / test depth, 0-10).

7.1 Proof of work and difficulty

	GnB-SMH	Bitcoin Core
Design	GnB-SMH v1: scrypt-ROMix structure over single-permutation SHAKE-256, 64 MiB, 2^20 dependent Keccak-f[1600] calls; ASERT (aserti3-2d construction) anchored at genesis, 2 h half-life, 30 s spacing; pow limit `0x2100ffff`	SHA-256d; 2016-block retarget clamped to x4; pow limit with 32 spare bits; `PermittedDifficultyTransition`
Verified correct	Compact codec identical to Core on all 587,202,560 decodable-exponent values; ASERT bit-identical to the aserti3-2d fixed-point step on more than 350,000 cases; Rust = Python reference on 103 inputs; strict `out < target`; timestamps >= latest parent and <= now + 600 s bound easing to 5.96% and exclude timewarp	`DeriveTarget` / `CheckProofOfWorkImpl`; MTP rule; BIP-94 timewarp fix only on testnet4/regtest
Weaknesses	PoW runs on self-declared bits before cheap checks (NET-2); free blocks on deep forks (POW-3); work() differs from spec (POW-6); latent ASERT saturation for high anchors (AUD-1); difficulty tests masked by `test-pow-limit` (POW-5)	Mainnet timewarp still possible in consensus
Tests	gnb-smh 14 unit + 1 vector test, 7 Python vectors, 3 node pow tests; mutation score 82.6% (176/213)	pow_tests.cpp (15 cases), arith_uint256_tests.cpp, headers_sync_chainwork_tests.cpp; fuzz `pow`, `pow_transition`, `headers_sync_state`, `p2p_headers_presync`; feature_minchainwork.py, mining_basic.py
Score (GnB / Core)	6 correctness, 3 defensive, 3 tests	9 / 9 / 9

7.2 Chain selection, GhostDAG and reorganisation

	GnB-SMH	Bitcoin Core
Design	GhostDAG k = 10 with reachability index; selected-parent chain applied to the UTXO set; `MAX_REORG_DEPTH` = 1,000 and a local finality height	Longest most-work chain; `ActivateBestChain`, `ConnectTip` / `DisconnectTip`; no reorg-depth limit
Verified correct	Deterministic, parent-order-independent tie-breaks (proptests); atomic reorg WriteBatch with re-validation; integrity hash chain checked at boot; reachability self-check against brute force	`CheckBlockIndex` invariants; `InvalidateBlock` / `PreciousBlock`; `m_chainstate_mutex` + `cs_main`
Weaknesses	Validation against the local tip splits honest nodes (DAG-1); blue work ignores merged blues and merged blocks never reach the ledger (DAG-3); k-cluster violation (DAG-4); node-local reorg cap (DAG-6); unbounded per-block cost (DAG-7); no acceptance lock (DAG-10); non-atomic extension commit (DAG-11)	None material at the code level reviewed
Tests	~120 tests incl. 5 proptests; convergence tests bypass state validation; differential compares two implementations with the same bug	validation_chainstatemanager_tests.cpp (12), validation_block_tests.cpp (10-thread ordering); fuzz `block_index_tree`, `chain`, `connect_block`; feature_block.py, rpc_invalidateblock.py, feature_chain_tiebreaks.py
Score (GnB / Core)	2 / 4 / 4	9 / 9 / 9

GHOSTDAG adds coloring, reachability reindexing, mergesets and multi-parent templates. Kaspa bounds that surface with mergeset-size, parent-count, merge-depth and finality-point rules; GnB has none of them and, because merged blue work is not counted, gains none of the DAG's security benefit. The complexity is therefore scored as attack surface.

7.3 Block and transaction validation, UTXO set and storage

	GnB-SMH	Bitcoin Core
Design	UTXO model; txid excludes script_sig; Merkle over txids (Bitcoin construction); RocksDB column families; undo records	UTXO model with segwit witness commitment; LevelDB coins view with head-block markers
Verified correct	Checked sums (inputs, outputs, fees, subsidy + fees); unconditional duplicate-input check; same-block later-output spends blocked; duplicate-tx rule; one coinbase at index 0 with one output <= subsidy + fees; carryover snapshot verified (SHA3, 452,726 rows, 99,999,999,789,793 sat)	MoneyRange; BIP30/34; `IsBlockMutated`; `DisconnectBlock` with unclean handling; `ReplayBlocks`
Weaknesses	No BIP30/34 (VAL-2); same-block coinbase spend (VAL-3); witness malleation (CRY-1); non-atomic commit (DAG-11); size rules split (VAL-12); no 64-byte-tx rule (VAL-10)	Historical: CVE-2010-5139, CVE-2012-2459, CVE-2018-17144 (all fixed)
Tests	~110 targeted single-node regressions; sprint_v1_quickwins.rs compiled out; shipped-snapshot test ignored; no crash or fork-context tests	transaction_tests (16), coins_tests (15); tx_valid.json and tx_invalid.json; fuzz `transaction`, `block`, `coins_view`, `connect_block`, `utxo_total_supply`; feature_block.py, feature_dbcrash.py, p2p_mutated_blocks.py
Score (GnB / Core)	2 / 5 / 3	9 / 9 / 10

7.4 Signatures, sighash and addresses

	GnB-SMH	Bitcoin Core
Design	Hybrid ML-DSA-65 AND Falcon-1024 (`clean` integer variant), 4-byte suite envelope; SHA3 sighash with 16-byte domain, version and chain id; 160-bit address hash with 4-byte SHA3d checksum	ECDSA/Schnorr on secp256k1; legacy, BIP143 and BIP341 sighash; bech32/bech32m
Verified correct	AND semantics with exact-length parsing; strict ML-DSA hint and Falcon compact decoding; fail-closed RNG, no C fallback; seeded RNG scoped to key generation; official ACVP and round-3 vectors	Strict DER, low-S, NULLFAIL; sign-then-verify; secure allocator; exhaustive and constant-time library tests
Weaknesses	Lenient consensus verifier and no witness commitment (CRY-1); separable suites (CRY-2); no amount commitment, library-defined preimage (CRY-3); no zeroization (CRY-4); unchecked Falcon signer (CRY-5); network-blind checksum (CRY-7)	Classical-only signatures (threat-model difference, not scored)
Tests	~140 tests; 15 ACVP sigVer cases; 3 Falcon round-3 vectors; no pinned sighash vectors	sighash.json (500 vectors), script_tests.json (1,258); fuzz `signature_checker`, `script_interpreter`, `key`, `sighash_cache`
Score (GnB / Core)	7 / 6 / 4	9 / 9 / 9

7.5 Peer-to-peer networking and synchronisation

	GnB-SMH	Bitcoin Core
Design	libp2p gossipsub (signed, strict), directed request-response sync, Kyber768 + Ed25519 transport upgrade, PEX	Custom P2P with v2 encrypted transport (BIP324), headers-first sync, addrman, compact blocks
Verified correct	Decoders bounded before allocation (`with_limit`, per-field caps); directed sync rejects trailing bytes; handshake capped at 16 KiB; connection limits; SHA-256 message ids	Validation before relay; `Misbehaving`; headers presync; per-peer in-transit limits; inbound eviction
Weaknesses	Remote panic (POW-2); relay before validation (NET-2); orphan memory (POW-4); misattributed penalties (NET-4, NET-5); IBD latch (NET-6); O(N^2) amplification (NET-7); unbounded maps (NET-8, NET-10); eclipse exposure (NET-11)	None material at the code level reviewed
Tests	~430 tests incl. live swarms; no proptest, no coverage-guided fuzzing; message processor (main.rs) untested	63 p2p_*.py functional tests; fuzz `process_message`, `process_messages`, `addrman`, `connman`, `net`, `p2p_transport_serialization`, `bip324_cipher_roundtrip`
Score (GnB / Core)	2 / 5 / 4	8 / 9 / 9

7.6 RPC, explorer and web front end

	GnB-SMH	Bitcoin Core
Design	axum JSON-RPC on loopback with API key, rate limits and CORS; Python explorer API; Cloudflare Worker proxy; browser wallet (WASM)	libevent HTTP RPC with cookie or rpcauth; no explorer or browser wallet
Verified correct	Loopback default; key validation and constant-time compare; Host/Origin gate; body, concurrency and timeout caps; wallet CSP, COOP, frame-ancestors	Cookie file permissions, HMAC rpcauth, 250 ms auth-failure delay, work-queue limits
Weaknesses	Plaintext origin link with shared secret (WAL-2); explorer XSS sinks without CSP (WAL-3); API slowloris (WAL-4); full-chain scans (NET-12); unbounded metric labels (NET-13)	Not applicable for web surfaces
Tests	auth.rs 54 and sprint_m_auth 18 tests; no HTTP end-to-end, JS, Worker or explorer-API tests	rpc_bind.py, interface_http.py, rpc_whitelist.py; fuzz `http_request`, `rpc`
Score (GnB / Core)	4 / 6 / 4	9 / 9 / 9

7.7 Mempool and block production

	GnB-SMH	Bitcoin Core
Design	Fee-rate index with count and byte caps, first-seen, no orphans, no RBF; internal CPU miner and getblocktemplate	Cluster mempool, RBF, orphanage, package relay; weight-bounded BlockAssembler
Verified correct	O(log n) index with invariant checker; checked sums; spawn_blocking for verification; self-mined blocks go through the full acceptance path (fail-safe)	Standardness, removeConflicts, removeForReorg, Expire, TrimToSize, TestBlockValidity
Weaknesses	MEM-1, MEM-2 (Critical), MEM-3, MEM-4, MEM-5 (High), MEM-6, MEM-7	None material at the code level reviewed
Tests	14 tests on the data structure only; gossip admission untested	mempool_tests, miner_tests; fuzz `tx_pool`, `tx_pool_standard`, `tx_package_eval`, `rbf`, `package_rbf`, `txorphan`; 20 mempool_*.py
Score (GnB / Core)	3 / 4 / 3	9 / 9 / 10

7.8 Wallet and key management

	GnB-SMH	Bitcoin Core
Design	Hybrid PQ keys per index derived with SHA3; Argon2id + AES-256-GCM keystores; CLI, light wallet, WASM browser wallet	Descriptor wallets, BIP32, SQLite; SHA-512-iterated KDF + AES-256-CBC
Verified correct	Argon2id 256 MiB t=4 p=4, bounded untrusted KDF parameters, fresh nonces, authenticated v1 keyfiles, 0600 atomic writes, BIP39 vectors, prior audit findings CR-05/06/07/10 fixed	Values from own chain state, -maxtxfee, waste-based selection (BnB, CoinGrinder, knapsack, SRD)
Weaknesses	WAL-1, WAL-2 (High); WAL-3, WAL-4; WAL-5, WAL-6, CRY-4, CRY-7	KDF is not memory-hard
Tests	~153 Rust tests; no JS, Worker, Python or wasm32-target tests	83 wallet unit cases; 13 wallet fuzz targets (`crypter`, `coinselection_*`, `coin_grinder`, `scriptpubkeyman`, `wallet_create_transaction`, ...); 68 wallet_*.py
Score (GnB / Core)	6 / 7 / 5	7 / 8 / 9

7.9 Shielded pool, attestation and ledger v1 (inactive in consensus)

	GnB-SMH	Bitcoin Core
Status	Shielded spends rejected (`RejectAll`); attestation not used for trust; ledger v1 is a research prototype outside the node	No equivalents
Findings	COH-1 (Critical if enabled), COH-2 (latent), LED-1 (sound SMT; open items), ATT-1	-

	GnB-SMH	Bitcoin Core
Tests	coherence 43 tests (proofs mocked); ledger 12 Rust + 11 Python tests with 28 single-fault vectors	-

7.10 Build, CI and supply chain

	GnB-SMH	Bitcoin Core
CI	Python reference + standalone `rust/` crate only; no workspace build, clippy, deny, audit, fuzz, sanitizer or Miri job	26 CI configurations, lint suite, fuzzing with sanitizers, previous-release compatibility
Dependencies	cargo-audit: 11 advisories (5 not compiled, 6 reachable); deny.toml stale and failing; PQClean crates unmaintained upstream; dead blst/secp256k1 deps	Few vendored subtrees (secp256k1, leveldb, crc32c, minisketch) with guix reproducible builds
Positive	Pinned toolchain, exact PQ crate pins, vendored-source hash pins, yamux advisory fork with lockfile guard (both guards pass when run in isolation)	-
Score (GnB / Core)	1	10

8. Historical Bitcoin bug classes applied to GnB

Bitcoin's history provides a checklist of consensus and denial-of-service defect classes. The table records, for each, how Bitcoin Core defends against it today and whether GnB's code does.

Bug class	Bitcoin Core protection	GnB code	Status
Value overflow (CVE-2010-5139)	`MoneyRange` in CheckTransaction/CheckTxInputs/ConnectBlock	`checked_add` on inputs, outputs, fees, subsidy + fees (main.rs:2065,2723,2752; core:957); harness confirms detection	Protected (no MoneyRange, VAL-11)
Duplicate inputs (CVE-2018-17144)	tx_check.cpp duplicate-input set	Unconditional per-transaction set (main.rs:2704-2716); reorg path re-checks	Protected
Merkle duplicate tail (CVE-2012-2459)	`ComputeMerkleRoot(&mutated)`, BLOCK_MUTATED, no invalid caching	Same tree (harness: identical to rust-bitcoin); `validate_no_duplicate_txs`; no invalid-id cache	Protected
64-byte transaction ambiguity	`IsBlockMutated` rejects 64-byte txs	None (VAL-10)	Partial (light clients)
Witness / signature malleation	Strict DER, low-S, NULLFAIL; witness commitment	txid excludes script_sig but no witness commitment; lenient verifier (CRY-1)	Exposed
Duplicate coinbase (BIP30/BIP34)	BIP30 checks and BIP34 height	None (VAL-2)	Exposed
Coinbase maturity	`CheckTxInputs` for every spend	Bypassed for same-block spends (VAL-3); gossip uses DAG count (MEM-4)	Exposed
Timewarp	BIP-94 on testnet4/regtest only (mainnet still exposed)	Absolute ASERT, timestamps >= latest parent and <= now + 600 s	Protected (better than Core mainnet)
Compact target negative/overflow encodings	`DeriveTarget` flags	Identical accept set (exhaustive 587 M-value differential)	Protected
Low-difficulty header/block spam	Headers presync, MinimumChainWork, anti-DoS threshold	PoW on self-declared bits, orphan pool, free deep forks (NET-2, POW-3, POW-4)	Exposed
Consensus depending on a serialization library (cf. 2013 BDB fork)	Hand-specified serialization	Sighash body is bincode 2 serde output (CRY-3); block size measured in bincode (VAL-12)	Partial
Remote crash from peer data	Fuzzed message processing, checked arithmetic	Overflow panic on wire height (POW-2)	Exposed
Orphan / memory exhaustion (cf. CVE-2018-17145)	Bounded orphanage, per-peer limits	10,000 x ~1 MB orphan blocks admitted on self-declared bits (POW-4)	Exposed
Crash consistency of chainstate	DB_HEAD_BLOCKS marker + ReplayBlocks	Extension commit non-atomic, errors ignored (DAG-11)	Exposed
Eclipse attacks	addrman buckets, anchors, feelers, eviction	Arbitrary eviction, unsolicited dials (NET-11)	Exposed

9. Comparative security scorecard

Scores are 0-10 per category (10 = no material weakness found, strong preventive controls and deep tests). The weighted totals summarise code-level security only.

3.1 / 10	9.1 / 10
GnB-SMH (commit 125e9e5)	Bitcoin Core (master f13ae51)

Category	Weight	GnB	Bitcoin Core	Basis (code evidence)
Consensus correctness and cross-node determinism	25%	2.0	9.5	GnB: PoW, compact targets and ASERT are exact and deterministic (exhaustive and differential evidence), but blocks are validated against the local tip (DAG-1), coinbase uniqueness and same-block maturity are missing (VAL-2, VAL-3) and the reorg cap is node-local (DAG-6), so honest nodes can split permanently. Core: connect-time validation in parent context, BIP30/34, no local reorg cap; mainnet timewarp mitigation (BIP-94) is still limited to testnet4/regtest.
Network and RPC DoS resistance	20%	1.5	8.5	GnB: one frame panics the processor (POW-2); gossip is relayed before validation and the ~0.8 s PoW runs on self-declared bits (NET-2); orphan pool and deep forks are nearly free (POW-3, POW-4); honest peers are penalised (NET-4, NET-5). Decoders are well bounded and RPC binds to loopback. Core: validation before relay, cheap PoW first, headers presync, misbehaviour scoring, addrman, per-peer download limits.
Block and transaction validation, monetary integrity	10%	5.0	9.5	GnB: checked arithmetic everywhere, CVE-2010-5139, CVE-2012-2459 and CVE-2018-17144 classes covered, coinbase value and carryover verified; same-block coinbase spend, no witness commitment, no MoneyRange, size rules split across encodings. Core: CheckTransaction/CheckTxInputs/ConnectBlock with MoneyRange, mutation detection and witness commitment.
Mempool and block-production liveness	10%	1.5	9.0	GnB: one dust transaction or ~116 ordinary ones make every template invalid (MEM-1, MEM-2); conflicts never leave the pool (MEM-3); template parents can exceed MAX_PARENTS (DAG-2). Core: standardness before script checks, removeConflicts/removeForReorg/Expire, weight-bounded templates checked with TestBlockValidity.
Cryptographic implementation	10%	6.5	9.0	GnB: hybrid ML-DSA-65 AND Falcon-1024 with exact-length parsing, constant-time clean Falcon, fail-closed RNG, chain-id sighash, official ACVP and round-3 vectors; but consensus accepts witness malleation, suites are separable, sighash has no amount commitment and secrets are not zeroized. Core: libsecp256k1 with exhaustive and constant-time tests, strict encodings, BIP143/341 sighash, secure allocator. Post-quantum readiness is noted but not scored (it depends on assumptions about future adversaries, not on code).
Test depth and consistency testing	10%	4.0	9.5	GnB: ~900 tests incl. proptests, KAT/ACVP vectors and Python differentials, all passing; but the state layer and the binary-only message processor are untested, a test file is compiled out, the real ASERT path is disabled by feature unification, and 37 mutants survive in gnb-smh. Core: 900 unit cases, 285 functional scripts, data-driven vectors (tx_valid/invalid, sighash, script_tests).
Defensive coding, memory and arithmetic safety	5%	6.0	8.5	GnB: memory-safe Rust, #![forbid(unsafe_code)] in consensus crates, 3 unsafe sites in production (FFI), overflow-checks in release; but reachable overflow panics, swallowed write errors and no acceptance lock. Core: C++ mitigated by sanitizers, 1,696 thread-safety annotations, Assert/Assume/CHECK_NONFATAL discipline.
Fuzzing, CI, release engineering and supply chain	5%	1.0	9.5	GnB: CI never compiles the workspace; no fuzz harness (corpora only); deny.toml fails its own policy; reachable advisories ignored; vendored guards cannot run. Core: 26 CI configurations (ASan/UBSan/integer, MSan, TSan, valgrind, fuzz, tidy, cross-arch), 206 fuzz targets, lint suite.

Category	Weight	GnB	Bitcoin Core	Basis (code evidence)
Wallet and key management	5%	5.5	8.0	GnB: Argon2id 256 MiB and AES-256-GCM with authenticated parameters, 0600 atomic files, strict wallet CSP; but signers trust API-supplied values without a fee cap, the web origin link is plaintext, explorer pages lack CSP and escaping. Core: values from own chain state, amount-committing sighash, -maxtxfee, waste-based coin selection, secure allocator (weaker, GPU-friendly KDF).
Weighted total	100%	**3.12**	**9.07**	Weighted mean of the category scores. Each category score is the lead auditor's calibration of the reviewers' 0-10 component scores against the findings and positive controls in this report.

Comparative note. GnB-SMH and Bitcoin Core differ by about six points on this scale. The gap is not in cryptographic primitives or arithmetic, where GnB is competitive and in places ahead (memory-safe language, absolute-schedule difficulty that rules out timewarp, post-quantum signatures), but in the parts that took Bitcoin Core a decade of incidents to harden: connect-time validation in the correct chain context, anti-DoS ordering before expensive work, peer accounting, mempool maintenance, crash consistency, and an assurance pipeline (fuzzing, sanitizers, functional tests) that exercises all of it continuously. Closing the five Critical findings and building that pipeline are prerequisites for any further comparison.

10. Remediation roadmap

Phase	Items
Phase 0: before any value is at stake	POW-2 (checked arithmetic on peer integers, drop wire height); NET-2 (validate before relay, cheap checks before PoW, peer penalties); MEM-1 and MEM-2 (shared standardness, byte-budgeted and self-checked templates); DAG-1 (accept with stateless/contextual checks, validate transactions on connect)
Phase 1: consensus and liveness	VAL-2, VAL-3 (coinbase uniqueness and same-block maturity in one shared rule set); DAG-6, DAG-3, DAG-4 (objective finality, blue work over mergeset blues, exact GHOSTDAG); MEM-3, MEM-4, MEM-5; POW-3, POW-4, DAG-2, DAG-7 (anti-DoS work threshold, bounded parents and mergesets, no orphan admission on self-declared bits); DAG-10, DAG-11 (acceptance lock, one WriteBatch per block); NET-4 to NET-7
Phase 2: assurance	CI that builds and tests the workspace with and without `test-pow-limit`, clippy -D warnings, cargo-deny, cargo-audit; cargo-fuzz targets for every wire decoder, `NewBlock`/`NewTransaction` handling, compact/ASERT, mempool and GhostDAG insertion; a multi-node functional test framework (two nodes, conflicting siblings, reorgs, crash restarts); adopt `audit/consistency` in CI; pin sighash, ASERT and work vectors in the Python reference; re-enable `sprint_v1_quickwins.rs`
Phase 3: protocol hardening (hard fork)	Witness commitment and canonical witnesses (CRY-1); sighash v3 committing to spent amounts and scripts with a specified byte layout (CRY-3, WAL-1); suite binding (CRY-2); tagged Merkle hashing (VAL-10); network-binding address checksum (CRY-7); lower pow limit (POW-3); fix `work()` to the specification (POW-6)
Web and operations	TLS or authenticated tunnel to the explorer origin, fixed cache TTLs, HSTS (WAL-2); site-wide CSP and escaping (WAL-3); API timeouts and firewalling (WAL-4); pinned reproducible WASM (WAL-5); wallet fee ceilings (WAL-1); systemd sandboxing (OPS-3)

Appendix A. GnB test-module inventory

Every test module in the repository, with the number of test functions and the security or consistency property it checks. *Kind* is derived heuristically from each file's code: property (proptest or seeded random), vectors (official or pinned data files), differential (cross-implementation), corpus (seed replay), async (tokio), unit (none of these).

A.1 Inline unit-test modules (`#[cfg(test)]`)

Module	Tests	Kind	Security / consistency property checked
`crates/coherence-core/src/lib.rs`	12	vectors	Shielded primitives: determinism, Merkle round trips, inflation and forgery, count binding, position guard, nullifier-set order independence, domain separation
`crates/gnb-crypto/src/address.rs`	7	unit	Address format, checksum, length and prefix rejection
`crates/gnb-crypto/src/core/carryover.rs`	5	unit	Bloch Genesis-3 carryover conversion, vout fixes, duplicate and malformed rows (shipped-snapshot test ignored)
`crates/gnb-crypto/src/core/mod.rs`	13	vectors	Coinbase value (overflow rejected, exact limit, genesis, one output); difficulty and genesis PoW; chain-id sighash separation and cross-chain replay
`crates/gnb-crypto/src/core/tokenomics.rs`	5	unit	Subsidy schedule examples, issuance below cap, minimum fee
`crates/gnb-crypto/src/crypto/mod.rs`	35	vectors	Sign/verify round trips, seed determinism, diversified addresses (12); Falcon canonical vs padded encoding, native-FP symbol tripwire, clean seeded KAT (4); KAT module: primitive offsets, legacy/magic ambiguity, strict verifiers, ML-DSA goldens, AND-combiner tamper matrix, hedging, suite isolation, no-panic fuzz loops (19)
`crates/gnb-crypto/src/hd_wallet/mod.rs`	34	vectors	HD wallet round trip, v1/v2 compatibility, derived-key authentication (17); recovery limits, preflight, import consistency (17)
`crates/gnb-crypto/src/types/heights.rs`	9	unit	Height newtype arithmetic
`crates/gnb-crypto/src/util.rs`	6	vectors	Bounded file reads; atomic 0600 writes
`crates/gnb-crypto/src/wallet/cli.rs`	10	vectors	CLI amounts, network, UTXO rows, canonical flags
`crates/gnb-crypto/src/wallet/client.rs`	12	async, vectors	RPC client input, HTTP budget, redirect refusal, envelope and id checks (2 tests are tautological)
`crates/gnb-crypto/src/wallet/disclosure.rs`	13	unit	Disclosure bundles: tamper, splice, canonical encoding, no secrets
`crates/gnb-crypto/src/wallet/encryption.rs`	22	unit	Keyfile round trips, wrong password, AAD/version tamper, length guards, password policy
`crates/gnb-crypto/src/wallet/http_rpc.rs`	5	unit	HTTP framing, slow-drip budget, error redaction
`crates/gnb-crypto/src/wallet/mod.rs`	40	vectors	Seed versions, sign/verify, build_tx change and network (16); legacy keystore KDF bounds, nonce shape, weak passwords, redaction, signed-message domain (14); transaction boundaries, coin-selection inputs, exact decimals, Bloch keystore compatibility (10)
`crates/gnb-crypto/src/wallet/seed.rs`	13	vectors	BIP39 official vectors, passphrase, V1 pinning
`crates/gnb-smh/src/difficulty.rs`	9	unit	Malformed compact bits rejected; round trip; round-down; work monotone; ASERT on-schedule, half-life, monotone fraction, clamps, extreme inputs
`crates/gnb-smh/src/lib.rs`	5	vectors, differential	Keccak step equals SHAKE-256; big-endian target compare; nonce/header sensitivity; log2_n bounds; canonical-parameter assert
`crates/gnb-wallet-wasm/src/lib.rs`	1	unit	Create, export, open and pay on the host (not on wasm32)
`crates/libp2p-yamux/src/lib.rs`	2	unit	Vendored yamux config (runs only outside the workspace)
`crates/pqcrypto-internals/src/lib.rs`	18	unit	Seeded-RNG scope cleanup on return and unwind, nesting, TLS teardown, determinism, fail-closed abort on RNG failure (runs only outside the workspace)
`node/src/analytics/mod.rs`	4	unit	Chain analytics helpers

Module	Tests	Kind	Security / consistency property checked
`node/src/attestation/mod.rs`	11	unit	Attestation verify() fails closed; nonce, digest, TEE and verity checks
`node/src/bin/gnb-cli.rs`	3	vectors	Send-command input validation
`node/src/coherence/mod.rs`	8	unit	Shielded pool validate/apply, exact undo, LIFO order, anchor bound, atomic double-spend rejection
`node/src/coherence/verifier.rs`	10	unit	Fail-closed RejectAll verifier, public-value binding, vkey pin, mock-prover refusal
`node/src/consensus/mod.rs`	20	property	GhostDAG basics, tips, error paths (15) and proptests: determinism, parent-order independence, blue-score monotonicity, topological order on DAGs of <= 17 blocks (5)
`node/src/consensus/reachability.rs`	4	unit	Ancestry queries, persistence round trip, delta drain
`node/src/dandelion.rs`	7	unit	Dandelion stem/fluff logic (module has no callers)
`node/src/main.rs`	19	async, vectors	Address parsing; submission gating (structure, self-mined, deep-reorg tip poisoning, below-finality fork-loser, bits); intra-tx duplicate inputs, minimum fee, template fee inflation; out-of-order same-block spends
`node/src/metrics/mod.rs`	8	unit	Prometheus metric registration and labels
`node/src/network/mod.rs`	28	async, vectors	Wire decode bounds, penalties, PEX handling, connection limits
`node/src/network/pex_validator.rs`	1	vectors	Peer-exchange address validation
`node/src/network/sync_rr.rs`	21	unit	Directed sync codec bounds, trailing-byte rejection, request limiter, pull-peer selection
`node/src/pow/mod.rs`	3	unit	Mine then verify; malformed bits; stop flag
`node/src/reorg.rs`	6	unit	Column families exist, write-error propagation, depth-cap boundary, batch atomicity
`node/src/rpc/auth.rs`	54	unit	API-key validation, constant-time compare, Host/Origin gate, rate limiting, IPv6 /64 aggregation
`node/src/rpc/mod.rs`	20	async	RPC handlers, CORS layer, heavy-read gate
`node/src/storage/mod.rs`	11	unit	Address-history index and snapshot rows (no UTXO/undo atomicity tests)
`node/src/sync/parent_fetch.rs`	7	unit	Orphan-parent fetch scheduling, backoff and give-up
`node/src/transport/stream.rs`	4	unit	KyberStream framing and limits
`rust/src/ledger.rs`	9	vectors, differential	Ledger v1: FIPS-202 fast path, random SMT operations vs proofs, non-canonical and truncated proofs, limits, wire strictness
`rust/src/lib.rs`	5	vectors, differential	Duplicate of the gnb-smh unit tests in the standalone crate (the only Rust tests CI runs)

A.2 Integration test files

Module	Tests	Kind	Security / consistency property checked
`crates/coherence-core/tests/fuzz_robustness.rs`	1	corpus	80,000 random Merkle-path verifications never panic
`crates/coherence-core/tests/known_unsafe_v1_authorization.rs`	4	unit	Characterises the known CR-01 authorization gap and the duplicate-leaf mitigation
`crates/gnb-crypto/tests/acvp_mldsa.rs`	3	vectors	Official NIST ACVP ML-DSA-65 sigVer cases (15) and keyGen tc26 interoperability
`crates/gnb-crypto/tests/falcon_submission.rs`	2	vectors	Official Falcon round-3 verify vectors; hybrid signature with an official Falcon half
`crates/gnb-smh/tests/vectors.rs`	1	vectors, differential	7 normative Python vectors with intermediates (cross-implementation)
`crates/libp2p-yamux/tests/lockfile_guard.rs`	2	unit	Exactly one yamux >= 0.13.10 in Cargo.lock (cannot run in place)
`crates/pqcrypto-internals/tests/vendor_pin.rs`	2	unit	SHA-256 pins of every vendored PQClean C file and exact file set (cannot run in place)

Module	Tests	Kind	Security / consistency property checked
node/tests/announce_then_pull.rs	1	async	Live two-swarm directed pull
node/tests/backfill_flood_lab.rs	1	unit	Measurement lab for deep-backfill insertion cost (618 s)
node/tests/coherence_block_serde.rs	3	unit	Shielded transactions survive the block wire format and are Merkle-bound
node/tests/coherence_invariants.rs	1	unit	Property test over 2,000 random blocks: no double spend, atomicity, consistency (proof check mocked)
node/tests/dst_harness.rs	5	async	Deterministic-simulation skeleton; equivocation on a bare GhostDAG
node/tests/frontier_sync.rs	3	unit	Deterministic simulation of the IBD release gate
node/tests/ghostdag_differential.rs	11	unit	Legacy (bounded BFS) vs fast interval-index coloring produce byte-identical GhostdagData
node/tests/ghostdag_replay_snapshot.rs	3	unit	Replay of a real Bloch snapshot (all ignored: needs external data)
node/tests/kat_address.rs	2	vectors	Self-generated address vectors
node/tests/kat_falcon1024.rs	4	vectors	Falcon-1024 sizes, round trip, rejection, malformed input
node/tests/kat_mldsa65.rs	4	vectors	FIPS 204 sizes, seeded golden, rejection, malformed input
node/tests/mempool_index_prop.rs	3	property	Seeded random operation sequences preserve fee-rate index invariants and ordering
node/tests/parser_corpus_floor.rs	6	corpus	Replays the node/fuzz seed corpora through the parsers (no coverage-guided fuzzing exists)
node/tests/peer_addr_churn_lab.rs	5	async, vectors	Peer-address graylist mechanics (3 live)
node/tests/reachability_persistence.rs	3	unit	Durable reachability index: reload and rebuild equivalence, migration
node/tests/reorg_hardening.rs	8	unit	Reorg aborts and restores on invalid fork blocks; A->B->A byte symmetry; mempool re-injection counts
node/tests/security_audit.rs	17	vectors	Coinbase value, maturity helper, dust, duplicate-transaction Merkle (CVE-2012-2459), PoW/bits/timestamp checks
node/tests/sighash_security.rs	2	unit	Sighash commits to outputs, outpoints and input index (relative comparisons); txid excludes script_sig
node/tests/sprint1b_storage_migration.rs	7	unit	Block storage round trip; corrupt bytes yield errors
node/tests/sprint1d_tx_wire.rs	6	unit	Transaction wire format round trip
node/tests/sprint_a1_transport_tests.rs	20	unit	ML-DSA handshake and AEAD primitives (the handshake is not used by the live transport)
node/tests/sprint_a2_stream_tests.rs	5	unit	KyberStream framing round trip
node/tests/sprint_a2_upgrade_tests.rs	5	unit	libp2p upgrade integration (2 tamper tests ignored)
node/tests/sprint_b6_hybrid_sig.rs	2	unit	Hybrid signature end to end through script_sig; both halves required
node/tests/sprint_bb_merkle_newtype.rs	7	unit	MerkleRoot newtype: serialization invariance and Merkle basics
node/tests/sprint_dd_mined_undo.rs	2	unit	Locally mined blocks write undo data and can be rolled back
node/tests/sprint_ee_convergence.rs	3	unit	Bare GhostDAG converges under different arrival orders
node/tests/sprint_ee_transport_convergence.rs	3	async, vectors	Live two-node convergence, equivocation and inbound cap
node/tests/sprint_gg_pre_ibd_mining.rs	4	unit	is_syncing gating of the miner before IBD
node/tests/sprint_k_address_tests.rs	9	unit	Strict checksummed address parsing
node/tests/sprint_m_auth_tests.rs	18	unit	RPC authentication decisions and rate limits
node/tests/sprint_n_mempool_tests.rs	11	unit	Mempool::add structural checks: duplicate, empty, conflict, oversize, coinbase, removal (no signatures)
node/tests/sprint_p_pex_tests.rs	24	unit	PEX validator, limiter and cap
node/tests/sprint_s_crypto_sizes.rs	4	unit	Hybrid size constants are consistent
node/tests/sprint_t1_seed_determinism.rs	5	unit	Seeded key generation is stable and does not leak into later random key generation
node/tests/sprint_u1_undo_data.rs	9	vectors	Undo-record CRUD and encoding

Module	Tests	Kind	Security / consistency property checked
`node/tests/sprint_u2_rollback.rs`	10	unit	rollback_block restores exact prior UTXO state
`node/tests/sprint_u3_reorg.rs`	16	unit	Common-ancestor search, reorg plan shapes, execution
`node/tests/sprint_u4_reorg_e2e.rs`	5	unit	Disposition logic end to end (without transaction validation)
`node/tests/sprint_v1_quickwins.rs`	7	unit	Disabled: the whole file is compiled out with #![cfg(any())]
`node/tests/sprint_y_integrity_chain.rs`	11	unit	Hash-chained integrity of persisted GhostdagData detects tampering
`node/tests/sync_frontier.rs`	18	unit	Frontier reconciliation unit tests
`node/tests/sync_locator.rs`	8	unit	Block locator construction
`node/tests/sync_peer_state.rs`	13	unit	Per-peer chain-state table (pins the monotonic hints exploited in NET-6)
`node/tests/sync_wire.rs`	7	vectors	GetTips/Tips wire frames and shared bounds
`node/tests/wire_decoder_fuzz.rs`	3	corpus	SplitMix-generated inputs (20,000 cases) never panic the block/tx/header decoders
`node/tests/wire_roundtrip_props.rs`	2	unit	Pseudo-random round trips of consensus wire formats (not proptest)
`rust/tests/ledger_vectors.rs`	3	vectors, differential	Rust vs Python ledger vectors: 6 SMT steps, 3 proofs, 5 valid blocks, 23 single-fault invalid blocks with error codes
`rust/tests/vectors.rs`	1	vectors, differential	Python PoW vectors against the standalone crate

A.3 Python reference tests

Module	Tests	Kind	Security / consistency property checked
`reference/check_vectors.py`	7	vectors, differential	Python reference reproduces the 7 PoW vectors
`reference/test_ledger.py`	11	vectors, differential	Python ledger vectors, random SMT operations, limits, producer-vs-validator differential over 25 random blocks

A.4 Audit consistency harness (this audit)

Module	Tests	Kind	Security / consistency property checked
`audit/consistency/tests/amount_consistency.rs`	5	property	CVE-2010-5139 overflow detection; subsidy schedule within allocation; fee function totality
`audit/consistency/tests/asert_differential.rs`	8	property	ASERT vs the aserti3-2d (BCHN) fixed-point step; bounds, monotonicity, half-life, extreme inputs, saturation finding AUD-1
`audit/consistency/tests/compact_core_vectors.rs`	3	vectors	Bitcoin Core bignum_SetCompact and CheckProofOfWork vectors applied to GnB
`audit/consistency/tests/compact_differential.rs`	5	property	Exhaustive 587,202,560-value decode differential vs Core SetCompact/DeriveTarget; encode vs Core GetCompact and rust-bitcoin (proptest)
`audit/consistency/tests/merkle_consistency.rs`	4	property	Merkle root vs rust-bitcoin and Core; CVE-2012-2459 collision; block id excludes signatures
`audit/consistency/tests/pow_check_semantics.rs`	7	property	Strict PoW boundary vs Core's <=; verify(); chain work vs Core GetBlockProof
`audit/consistency/tests/pow_reference_differential.rs`	3	property, vectors, differential	GnB-SMH vs normative Python on 96 random cases; scratchpad reuse; per-thread pad

A.5 Audit proof-of-concept tests (this audit, in `audit/poc`)

Module	Tests	Kind	Security / consistency property checked
`audit/poc/crates/coherence-core/tests/audit_poc_coherence.rs`	3	unit	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control

Module	Tests	Kind	Security / consistency property checked
`audit/poc/crates/gnb-crypto/tests/audit_poc_crypto.rs`	11	vectors	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/crates/gnb-crypto/tests/audit_poc_pow.rs`	5	unit	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/crates/gnb-crypto/tests/audit_poc_wallet.rs`	6	unit	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/crates/gnb-smh/tests/audit_poc_pow.rs`	10	vectors	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/node/tests/audit_poc_dag.rs`	9	unit	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/node/tests/audit_poc_mempool.rs`	5	unit	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/node/tests/audit_poc_net.rs`	13	async	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control
`audit/poc/node/tests/audit_poc_validation.rs`	11	unit	Reviewer proof-of-concept: each passing test demonstrates a finding or confirms a control

Appendix B. Bitcoin Core security and consistency test modules referenced

All paths are relative to the Bitcoin Core tree at `f13ae51` and were checked to exist.

Category	Modules
Unit tests (src/test)	pow_tests.cpp (15 cases), arith_uint256_tests.cpp (`bignum_SetCompact`), headers_sync_chainwork_tests.cpp, merkle_tests.cpp, sighash_tests.cpp + data/sighash.json, script_tests.cpp + data/script_tests.json, transaction_tests.cpp (16) + data/tx_valid.json, data/tx_invalid.json, coins_tests.cpp (15), validation_block_tests.cpp, validation_chainstatemanager_tests.cpp (12), blockchain_tests.cpp, txvalidationcache_tests.cpp, mempool_tests.cpp, miner_tests.cpp, denialofservice_tests.cpp, addrman_tests.cpp, net_tests.cpp, key_tests.cpp
Wallet unit tests (src/wallet/test)	coinselector_tests.cpp and 17 further files (83 cases)
Fuzz targets (src/test/fuzz, 206 in total)	`pow`, `pow_transition`, `headers_sync_state`, `p2p_headers_presync`, `process_message`, `process_messages`, `addrman`, `connman`, `net`, `node_eviction`, `p2p_transport_serialization`, `bip324_cipher_roundtrip`, `block`, `block_header`, `block_index`, `block_index_tree`, `chain`, `connect_block`, `coins_view`, `coinscache_sim`, `utxo_total_supply`, `utxo_snapshot`, `transaction`, `decode_tx`, `merkle`, `tx_pool`, `tx_pool_standard`, `tx_package_eval`, `ephemeral_package_eval`, `rbf`, `package_rbf`, `txorphan`, `txorphan_protected`, `txorphanage_sim`, `signature_checker`, `script_interpreter`, `sighash_cache`, `key`, `crypter`, `coinselection_bnb`, `coinselection_knapsack`, `coinselection_srd`, `coin_grinder`, `scriptpubkeyman`, `wallet_create_transaction`, `http_request`, `rpc`, `addition_overflow`, `multiplication_overflow`
Functional tests (test/functional, 285 scripts)	feature_block.py, feature_minchainwork.py, feature_dbcrash.py, feature_chain_tiebreaks.py, feature_assumeutxo.py, mining_basic.py, p2p_invalid_block.py, p2p_invalid_tx.py, p2p_invalid_messages.py, p2p_unrequested_blocks.py, p2p_headers_sync_with_minchainwork.py, p2p_mutated_blocks.py, p2p_ibd_stalling.py, p2p_addr_relay.py, p2p_eviction.py, p2p_orphan_handling.py, mempool_reorg.py, mempool_expiry.py, mempool_limit.py, mempool_spend_coinbase.py, mempool_dust.py, rpc_invalidateblock.py, rpc_bind.py, interface_http.py, and 68 wallet_*.py
CI configurations (ci/test)	00_setup_env_native_asan.sh (address, integer, undefined, float-divide-by-zero), _msan, _tsan, _valgrind, _fuzz, _fuzz_with_msan, _fuzz_with_valgrind, _tidy, _iwyu, _previous_releases, _nowallet, _alpine_musl, _chimera_lto, arm, s390x (big-endian), i686, riscv, win64, macOS, FreeBSD/NetBSD/OpenBSD cross
Static analysis and lint	src/.clang-tidy (bugprone, performance, modernize, misc-no-recursion), test/lint (15 scripts: circular dependencies, includes, locale, shell, python, docs, subtrees)
secp256k1 subtree	tests.c (113 test groups), tests_exhaustive.c, ctime_tests.c (constant-time checks under valgrind)

Appendix C. Reproduction commands

Step	Command
GnB workspace tests	`cargo test --release --locked --workspace --lib --bins --test <each tracked test> --no-fail-fast`
Standalone crate	`cd rust && cargo test --release --locked`
Python reference	`python3 reference/check_vectors.py && python3 reference/test_ledger.py && python3 reference/gen_ledger_vectors.py && git diff --exit-code reference/ledger_vectors.json`
Consistency harness	`cd audit/consistency && cargo test --release -- --nocapture`
Mutation testing	`cargo mutants -p gnb-smh --file crates/gnb-smh/src/difficulty.rs --file crates/gnb-smh/src/lib.rs --output OUT`, then `python3 audit/consistency/scripts/mutants_vs_harness.py OUT/mutants.out WORKDIR`
Proof-of-concept tests	Copy `audit/poc/<crate>/tests/*.rs` into the matching crate's `tests/` directory, then `cargo test --release --locked -p <crate> --test <name>` (see audit/poc/README.md)
Dependency audit	`cargo audit --db <advisory-db> --no-fetch`; `cargo deny --offline check bans licenses sources`
Bitcoin Core metrics	`git clone --depth 1 https://github.com/bitcoin/bitcoin` at `f13ae519de94394314cd63a27924b0998ac65d45`; counts by grep over src/test, src/test/fuzz (`FUZZ_TARGET`), test/functional and ci/test
This report	`python3 audit/report/gen_inventory.py && python3 audit/report/gen_content.py && python3 audit/report/build_report.py && node audit/report/print_pdf.cjs audit/report/out/report.html audit/GnB-SMH-Code-Audit-2026-10-14.pdf`